

MOHID Support Automation V1

Problem and First Implementation Living Specification

Companion to the Governed Support Automation Platform Living Reference

Document role	Living product and engineering build document
Release target	MOHID V1 governed support automation pilot
Document version	0.6 active baseline
Status	Active; one of two living project documents
Owner	Akbar Aman
Date	July 27, 2026
Human control plane	SON - Support Operations Navigator
Reference vision	Governed Support Automation Platform - Living Vision and Working Reference v1.5

Canonical V1 objective

Test whether MOHID’s authoritative HelpDocs can support governed automation of repetitive support requests, while routing uncertain or restricted cases to the appropriate humans and preserving a complete case-level record of evidence, automated actions, human decisions, and final outcomes.

Automate what is safe. Route what requires judgment. Audit the full lifecycle.

Document Control

Purpose	Define MOHID's support problem and first implementation, including the company-aligned technology stack, product requirements, interfaces, tests, rollout, and evidence needed to expand or stop.
Audience	Product owner, implementers, reviewers, MOHID stakeholders, security and operations partners, and future maintainers.
Authority	This paper and the companion Platform Living Reference are the only active project documents. This paper may make MOHID implementation choices but may not weaken the platform doctrine.
Change rule	This is a living specification with stable filename and numbered internal revisions. Changes must update the decision register, affected acceptance criteria, and version history rather than create another competing paper.
Implementation stance	Build the first MOHID vertical slice on the technology stack approved by MOHID/DeenTek. Keep provider-neutral internal contracts, implement one approved adapter per boundary, and avoid premature multi-provider parity.
Engineering method	Use test-driven delivery: acceptance and contract tests define behavior before production code; integration and replay tests gate every provider-backed workflow.
Current external-action rule	No customer-facing automatic action is enabled until offline replay, shadow mode, and universal human approval have passed their gates.

Version History

Version	Date	Status	Summary
0.1	July 27, 2026	Superseded baseline	First executable V1 specification derived from Vision v1.0, v0.1/v0.2 engineering foundations, and earlier MOHID evidence.
0.2	July 27, 2026	Superseded consolidated baseline	Established the two-document source-of-truth model and incorporated repository-grounded technical history.
0.3	July 27, 2026	Superseded baseline	Narrowed the earlier implementation to a technology-stack baseline and made company-stack alignment, test-driven delivery, and provider-neutral contracts the first-implementation doctrine.
0.4	July 27, 2026	Superseded baseline	Bound MOHID to the shared-runtime/application-package contract with immutable run versions and a declarative, non-bypassable behavior package.
0.5	July 27, 2026	Superseded baseline	Grounded the first slice in Fahad's live-test logs, made mTAP the initial proof, formalized the knowledge compiler, and replaced the ten-day promise with acceptance-gated work packages.
0.6	July 27, 2026	Active baseline	Clarifies API and MCP ownership, adds protocol-neutral knowledge-source and query seams, separates knowledge writes from operational actions, and explicitly defers generic MCP infrastructure without expanding the mTAP proof.

Statement Types

[CONFIRMED]	Stable fact supported by a direct owner correction, repository evidence, or clearly confirmed MOHID history.
[V1 DESIGN]	Current implementation choice for V1. It may change through the decision register without weakening the Platform Living Reference.
[HYPOTHESIS]	Claim the pilot must test rather than assume.
[OPEN]	Missing choice that must be resolved before its stated rollout gate.
[DEFERRED]	Intentionally excluded from V1.

Authoring Basis

Sources Found and Applied

1. **Governed Support Automation Platform - Living Vision and Working Reference v1.5.**
2. **GitHub legacy-prototype-v1**, audited on the **main** branch through commit **6eba0ff75d92** (March 3, 2026), as the technical baseline for the earlier stack, reusable fixtures, and regression evidence.
3. **Client Feedback: RAG System & Email Integration**, including Fahad Faridi's March 25, 2026 live-test inputs and verbatim Render logs, as direct regression and provider-failure evidence.
4. **Governed Support Resolution OS v0.2** and **Evidence-Backed Support Resolution Platform v0.1** as superseded engineering and design-history sources.
5. **Authoring handoffs and research conversations** as historical reasoning that has now been reconciled into the two living documents.
6. **Mohid Production Phase, Mohid Tier 1 Parts 1–3**, and **MOHID project details** as historical client facts, call notes, earlier implementation behavior, and failure evidence.

Repository-generated Markdown is not treated as code truth when it conflicts with the audited implementation. Current code, requirements, README, and commit history take precedence only for facts about the earlier stack.

Source Hierarchy

When sources conflict, apply this order:

1. direct corrections and decisions from Akbar Aman incorporated into the current numbered revisions;
2. the Platform Living Reference for product doctrine and reusable boundaries;
3. current accepted and provisional decisions recorded in this paper;
4. the audited GitHub repository for facts about the earlier technology stack and regression fixtures;
5. v0.2 and v0.1 for superseded but reusable engineering concepts; and
6. historical chats, call notes, and repository prose for context and lessons, never mandatory architecture.

Authoritative Current-State Decisions

- The product is a configurable support-resolution platform, not a chatbot, reply generator, reviewer dashboard, replacement help desk, or general agent framework.
- **legacy-prototype-v1** is the earlier standalone MOHID implementation and is used only as a technical baseline, fixture source, and regression reference.
- Automatic resolution remains in V1 and may be enabled only for evaluated low-risk categories.
- Resolution, dispatch, and routing are separate typed decisions.
- SON is the required human control plane.
- The Support Case is the human-facing unit of work; a case may contain multiple Request Runs.
- The HelpDocs site is the authoritative MOHID knowledge source.
- Official HelpDocs API or export access remains unconfirmed and client-gated. Approved HTML or fixture acquisition is sufficient for the first mTAP proof, provided it passes the same semantic source contract and knowledge-compiler controls.
- Fahad's mTAP live tests are the first regression baseline; mTAP is the first bounded knowledge slice, and KIOSK is the first reuse test after the mTAP gate passes.

- Retrieval, evidence qualification, verification, policy, dispatch, routing, human actions, tool execution, feedback, and reopening must share one case record.
- The first integrated implementation uses MOHID/DeenTek-approved services: Gemini through the company's Google Cloud environment, Office 365 for the support mailbox, Mandrill/Mailchimp Transactional for outbound email, and an approved MOHID server or Google Cloud deployment target.
- Product contracts remain provider-neutral even though the first adapters are company-specific.
- Test-driven delivery is mandatory: behavior is specified by acceptance and provider-contract tests before production implementation.
- V1 moves progressively through historical replay, shadow mode, universal approval, and narrow automatic resolution.

Stale Assumptions Excluded

- **One-primary-article RAG** is a documented earlier failure mode, not the V1 architecture.
- **Flattened text plus fixed character chunks** is legacy ingestion, not the production knowledge model.
- **Similarity thresholds as factual confidence** are excluded; retrieval score alone never authorizes dispatch.
- **FAISS, Render, Mailgun, and the Gemini–Mistral–OpenAI fallback chain** are actual legacy implementation assets, not permanent dependencies.
- **Flat JSON runtime state** is not acceptable for production cases, review decisions, or audit events.
- **Every request is answer generation** is false. Clarification, escalation, rejection, routing, and no action are first-class outcomes.
- **Universal human approval as the end state** is false. It is a rollout stage and configurable tenant policy.
- **The legacy customer-confirmation link as a universal escalation requirement** is excluded. Customer confirmation may be retained for selected actions if MOHID approves it, but it is not platform or V1 doctrine.
- **Broad self-service multi-tenancy, voice, kiosk, billing, connector marketplaces, and employee scoring** are deferred.
- **A generalized MCP integration framework, production MCP server, live HelpDocs querying, and HelpDocs write-back** are deferred; protocol availability alone does not justify them.

Final Document Outline

This paper moves from the MOHID problem and technology baseline to executable contracts: hypotheses; users and permissions; lifecycle; domain model; knowledge ingestion; retrieval and evidence; policy and decisions; deterministic tools; SON screens; persistence and audit; security; evaluation; rollout; company-aligned stack; test-driven delivery; APIs; build order; acceptance; risks; and the immediate acceptance-gated vertical-slice plan.

Contents

Document Control	i
Authoring Basis	iii
1 Executive Product Summary	1
1.1 V1 Success Definition	1
1.2 V1 Falsification Conditions	1
2 Relationship to the Platform Living Reference	1
2.1 MOHID Application Binding	2
3 Confirmed MOHID Problem and Technology Baseline	2
3.1 Stable Facts	2
3.2 Earlier-to-First-Implementation Technology Stack	3
3.3 Regression Evidence Retained	4
4 V1 Objective, Hypotheses, and Evidence	4
4.1 Primary Hypotheses	4
4.2 Evidence Required	5
5 V1 Scope and Non-Scope	5
5.1 In Scope	5
5.2 Explicitly Deferred	6
6 Users, Roles, Teams, and Permissions	6
6.1 Initial Queue Model	7
6.2 Permission Invariants	7
7 End-to-End Case Lifecycle	7
7.1 Case and Run Semantics	7
7.2 Canonical States	8
7.3 Failure and Recovery States	9
7.4 Transition Rules	9
8 Domain Model and Contracts	9
8.1 Relationship Model	9
8.2 Required Objects	9
8.3 Canonical Decision Bundle	11
8.4 Contract Rules	11
9 HelpDocs Ingestion and Knowledge Lifecycle	11
9.1 Production Knowledge Model	11
9.2 Acquisition Boundary and Initial Adapter	12
9.3 Ingestion Pipeline	12
9.4 Canonical Document Contract	12
9.5 Generated Markdown and Parser Discipline	13
9.6 Chunking Rules	13
9.7 Synchronization and Deletion	13
9.8 Initial Implementation Choice	14
9.9 Ingestion Acceptance Criteria	14
10 Retrieval and Evidence Packet Design	14
10.1 Retrieval Sequence	14
10.2 Live Knowledge Query Boundary (Deferred)	15
10.3 Intent Decomposition	15
10.4 Corrections, Replies, and Reopening	15

10.5 Evidence Qualification	15
10.6 Evidence Packet Contract	15
10.7 Conflict and Insufficiency Behavior	16
10.8 Retrieval Evaluation	16
11 Resolution, Verification, Policy, Dispatch, and Routing	16
11.1 Resolution	16
11.2 Verification	17
11.3 Dispatch	17
11.4 Routing	17
11.5 Automatic-Dispatch Gate	17
11.6 Policy Representation	18
12 Tool Execution and External Integrations	18
12.1 Capability and Authorization Separation	18
12.2 V1 Tool Catalog	18
12.3 ToolAction Contract	19
12.4 Execution Invariants	19
12.5 Email Integration Sequence	19
12.6 Integration Preconditions	20
13 SON V1 Product Requirements	20
13.1 Product Surface	20
13.2 Screen 1: Case Queue	20
13.3 Screen 2: Case Detail	20
13.4 Reviewer Action Requirements	20
13.5 Screen 3: Automatic-Resolution Review	21
13.6 Meaningful Review Requirements	21
14 Persistence, Audit, and Timing Model	21
14.1 Persistence Boundaries	21
14.2 Initial PostgreSQL Tables	21
14.3 AuditEvent Contract	21
14.4 Timing Intervals	22
14.5 Consistency and Recovery	22
15 Security, Privacy, and Tenant Boundaries	22
15.1 Security Objectives	22
15.2 Tenant and Data Isolation	23
15.3 Privacy and Retention	23
15.4 Security Release Gates	23
16 Evaluation Dataset and Metrics	24
16.1 Dataset Construction	24
16.2 Dataset Splits and Governance	24
16.3 Required Metrics	24
16.4 Provisional Engineering Gates	25
16.5 Failure Severity	26
17 Rollout and Pilot Plan	26
17.1 Kill Switch	26
17.2 Stop Conditions	27
17.3 Rollback	27
18 First Implementation Technology Stack and Engineering Method	27
18.1 Runtime and MOHID Package Boundary	27
18.2 Company-Aligned V1 Stack	27

18.3 Test-Driven Delivery Contract	28
18.4 Component Topology	29
18.5 Required Provider Interfaces	29
18.6 Environment Model	29
18.7 Repository Shape	30
19 API Surface	30
19.1 API Conventions	30
19.2 Required Endpoints	31
19.3 Manual Intake Example	31
19.4 Human Decision Example	31
20 Engineering Epics and Build Order	31
20.1 Critical Path	31
21 V1 Acceptance Criteria	36
21.1 Product and Workflow	36
21.2 Knowledge and Evidence	36
21.3 Governance and Safety	36
21.4 Evaluation and Operations	36
21.5 Automatic Resolution	37
22 Risks, Open Decisions, and Deferred Work	37
22.1 Risk Register	37
22.2 Open Decisions by Deadline	37
22.3 Deferred Backlog	38
23 Decision Register	38
24 Immediate Vertical-Slice Plan	40
24.1 Plan Rule and End-to-End Goal	40
24.2 Acceptance-Gated Work Packages	40
24.3 First-Slice Acceptance	41
24.4 Client Dependencies Beside the Plan	41
24.5 Work-Package Non-Goals	42
24.6 Expansion Trigger	42
25 Closing Build Contract	42
A GitHub Issue Template	43
B Initial Reason-Code Vocabulary	43

1 Executive Product Summary

[**CONFIRMED**] MOHID has strong owner-written HelpDocs, an email-oriented support workflow, routine questions that may be answered directly, cases that require forms or ticket actions, tiered escalation, and meaningful post-response feedback or reopening.

[**V1 DESIGN**] MOHID V1 is the first company-aligned implementation of the governed support solution. The earlier repository contributes only its technology baseline, reviewed fixtures, and regression cases; V1 behavior is defined by this specification and its tests.

V1 product definition

A configurable support-resolution system that automatically handles eligible MOHID cases, routes uncertain or restricted cases to the appropriate humans, and audits every AI action and human decision from intake through final outcome.

The V1 vertical slice is:

controlled intake → **versioned HelpDocs** → **Evidence Packet** → **resolution** → **verification** → **policy** → **dispatch** + **routing** → **SON or automatic action** → **outcome**

The first build does not need to replace MOHID's help desk or construct an enterprise platform. It begins with the mTAP slice from Fahad's own tests and must demonstrate that one real support workflow can be reconstructed, evaluated, reviewed, and eventually automated without hiding uncertainty or losing accountability. KIOSK is added next to test reuse, not breadth for its own sake.

1.1 V1 Success Definition

V1 succeeds when all of the following are true:

1. the authoritative HelpDocs corpus is reproducibly ingested, versioned, searchable, and citable;
2. representative historical tickets can be replayed against a fixed corpus and adjudicated labels;
3. every intent receives an explicit resolution, dispatch, and routing decision;
4. substantive output claims map to approved evidence spans;
5. reviewers can make meaningful decisions in SON with evidence, policy, and verification visible;
6. permitted tool actions are schema-validated, policy-checked, idempotent, and auditable;
7. universal-review operation is reliable enough to pilot; and
8. one or two low-risk categories may enter narrow automatic resolution only after their release gates pass.

1.2 V1 Falsification Conditions

The product thesis should be narrowed or stopped if authoritative HelpDocs consistently lack enough information, evidence qualification cannot reliably separate supported from unsupported cases, reviewer workload is not reduced, false automatic resolution remains unacceptable, routing cannot be operationalized, or the required integrations cannot be governed safely.

2 Relationship to the Platform Living Reference

The Platform Living Reference defines the durable destination, authorship boundary, and doctrine. This paper owns MOHID's problem definition, technology constraints, first-implementation requirements, tests, and changeable implementation detail.

Platform Living Reference owns	This MOHID living specification owns
Product category and purpose	MOHID release scope and build order
Canonical operating line	Domain objects, APIs, state transitions, and screens
Resolution/dispatch/routing separation	Concrete enums, fields, validation, and persistence
SON as human control plane	SON V1 queue, case detail, and review interactions
Unified AI-human accountability	Event types, timing intervals, and audit storage
Provider independence	Current adapters and likely deployment path
Anti-surveillance boundary	Authorized metrics, access rules, and UI exclusions
Progressive autonomy	Pilot phases, release gates, rollback, and kill switch

An implementation convenience may not weaken a platform invariant. If a desired change would collapse the three decision planes, hide evidence from reviewers, bypass deterministic authorization, infer employee motive, or make a provider a permanent product dependency, it requires a coordinated Platform Living Reference revision rather than an ordinary V1 decision record.

2.1 MOHID Application Binding

[V1 DESIGN] MOHID V1 is one dedicated application executed by the shared governed runtime. The current identifiers are `tenant_id=mohid` and `application_id=mohid-support`. The application is deployed independently in local, development, staging, or production; DeenTek remains the organizational and infrastructure context. Tenant and application identifiers stay distinct even if a later security design introduces a DeenTek parent tenant.

```
applications/mohid-support/
  manifest.yaml
  knowledge.yaml
  taxonomy.yaml
  routing.yaml
  autonomy.yaml
  providers.yaml
  templates/
  evaluations/
```

This package declares MOHID knowledge, terminology, routes, templates, autonomy rules, provider-profile references, and evaluation cases. It contains no credentials, provider SDKs, executable workflow code, or replacement policy gate. At RequestRun creation, the runtime validates the package and pins `runtime_version`, `application_package_version`, `workflow_version`, `corpus_version`, `policy_version`, `model_profile_version`, and `adapter_versions`. An incompatible package is rejected, and an update never changes an in-flight run.

3 Confirmed MOHID Problem and Technology Baseline

3.1 Stable Facts

- **[CONFIRMED]** MOHID is the first client product/application in this work; DeenTek is the broader company context rather than the name of the platform being built.
- **[CONFIRMED]** The HelpDocs website is the authoritative support knowledge source.
- **[CONFIRMED]** Support is substantially email-oriented and the production support mailbox is on Office 365.
- **[CONFIRMED]** MOHID already uses Google Cloud and provided Gemini access for this work.
- **[CONFIRMED]** MOHID uses Mailchimp/Mandrill for transactional outbound email.
- **[CONFIRMED]** The backend may call approved MOHID APIs or run on MOHID-managed servers; an approved Google Cloud target is also compatible with the company's environment.
- **[CONFIRMED]** Routine documented questions may be answered directly with links to relevant HelpDocs.

- **[CONFIRMED]** Some cases require ticket prefilling, a form, API action, forwarding, or escalation. Whether a customer must explicitly confirm a particular external action remains a MOHID workflow decision.
- **[CONFIRMED]** A Level 1 / Level 2 support path exists conceptually.
- **[CONFIRMED]** Negative feedback, reopening, or a failed outcome matters after dispatch.
- **[CONFIRMED]** Automatic dispatch was part of the original objective; reviewer assistance alone is insufficient.
- **[CONFIRMED]** The Render implementation processed Fahad’s real email-style requests end to end and produced outbound responses, proving the basic deployment and transport loop.
- **[CONFIRMED]** The same live tests exposed retrieval, context-selection, conversation-state, provider-quota, and delivery failures. They do not establish that the generated resolutions were correct or production-safe.

3.2 Earlier-to-First-Implementation Technology Stack

The earlier implementation is preserved at [legacy-prototype-v1](#). Its role in this paper is limited to documenting the previous stack and supplying tests and fixtures. The first implementation uses the company-aligned target below.

Layer	Earlier stack	First implementation target
Backend	Python, FastAPI, Docker	Python, FastAPI, Pydantic, and Docker retained behind versioned domain and provider contracts
Model	Direct Gemini, Mistral, and OpenAI calls with fallback behavior	Gemini through MOHID-owned Google Cloud access; one ModelProvider adapter and no runtime fallback chain unless explicitly approved
Knowledge	HelpDocs scraped to flattened text and fixed-size chunks	Approved HelpDocs acquisition into raw snapshots, canonical structured JSON, generated Markdown, and versioned search projections
Retrieval	FastEmbed embeddings and an in-image FAISS index	PostgreSQL plus pgvector on MOHID-approved infrastructure; lexical or reranking additions must earn inclusion through retrieval tests
Application data	Local JSON on an ephemeral filesystem	PostgreSQL as the durable source of truth for cases, decisions, events, policies, and evaluation records
Inbound email	Mailgun webhook	Office 365 shared mailbox through Microsoft Graph or an approved forwarding adapter
Outbound email	Mailgun send API	MOHID’s Mandrill/Mailchimp Transactional path through an OutboundMessageProvider adapter
Hosting	Render container	Docker on MOHID-managed servers or an approved MOHID Google Cloud target
Human interface	Lightweight chat and feedback pages	SON review surfaces using DeenTek’s approved web stack; Next.js/TypeScript is the fallback proposal only if no internal standard is required
Testing	Manual examples and endpoint checks	Pytest, testcontainers, provider contract suites, historical replay, security tests, and Playwright end-to-end checks written before implementation behavior

First-implementation rule

The integrated vertical slice is built against the company-approved stack from the start. Local fakes exist only to drive tests and preserve development speed. They may not establish a competing deployment path. Provider SDKs remain inside adapters, and the same contract suite must validate both each fake and the approved provider implementation.

3.3 Regression Evidence Retained

The legacy system is a baseline, not a discarded attempt. It proved that a deployed FastAPI service could ingest support requests, search HelpDocs, call a model, generate a response, and send email. Fahad’s March 25 live tests then supplied the first real regression evidence for what the production-oriented build must improve.

Observed baseline behavior	What the logs establish	Required V1 regression
Clean mTAP setup wording ranked the expected article first with a strong combined score. The same intent wrapped in a greeting and realistic email phrasing kept the article first but materially weakened ranking signals.	Retrieval was sensitive to surface wording even when intent was unchanged.	Paired clean/noisy cases must be evaluated together; normalization and retrieval must preserve the expected evidence and report any material rank or coverage degradation.
Only the primary article entered generation context while other retrieved articles were labeled “helpful sources.”	Search could find relevant material that the answer path never used.	Every required source must be eligible for the Evidence Packet; displayed-only sources cannot satisfy evidence coverage.
“This is not the right answer” was searched as a new standalone query and matched unrelated financial content.	Thread identity existed, but corrective feedback did not drive a case-aware retry or reopening path.	A correction becomes a new Request Run on the existing Support Case, bound to prior request, response, evidence, and feedback.
Gemini repeatedly returned quota errors and Mistral completed the request.	Model availability and response quality were separate concerns hidden inside an implicit fallback chain.	Provider quota/outage is recorded as its own failure class; no fallback runs unless explicitly configured and versioned by policy.
Mailgun sometimes returned a delivery error and later succeeded.	Channel delivery could fail independently of retrieval or drafting.	Delivery receipts, retries, and reconciliation are measured separately from answer quality; no send is marked successful without a provider receipt.
KIOSK, donation receipt, and program-registration emails produced plausible but incomplete or mismatched evidence.	Real tickets include feature questions, exact procedures, ambiguity, and knowledge gaps.	These cases become later slices for reuse, procedural completeness, clarification, and escalation after mTAP passes.

First proof slice

mTAP is the first bounded implementation slice. It has a known clean/noisy pair, a known expected article, related setup and hardware documents, provider-failure traces, and a correction-message failure. KIOSK follows as the first materially different slice to prove reuse without rewriting the runtime.

4 V1 Objective, Hypotheses, and Evidence

4.1 Primary Hypotheses

- [HYPOTHESIS] Structured, versioned HelpDocs preserve enough procedural detail to improve evidence retrieval over flattened text.

2. **[HYPOTHESIS]** Per-intent evidence qualification can distinguish safe documented cases from requests requiring clarification or escalation.
3. **[HYPOTHESIS]** A prepared Evidence Packet and verification summary reduce reviewer investigation and editing time without turning review into blind approval.
4. **[HYPOTHESIS]** A policy gate can identify a narrow set of low-risk cases for automatic dispatch with an acceptably low false-resolution rate.
5. **[HYPOTHESIS]** One case-level audit history creates operational value beyond a generated reply by exposing routing, delays, tool results, corrections, and final outcomes.

4.2 Evidence Required

Each hypothesis must be evaluated with:

- versioned historical tickets and adjudicated expected decisions;
- retrieval and Evidence Packet measurements;
- claim and procedure verification results;
- reviewer actions, edits, override reasons, and handling time;
- dispatch, tool, feedback, and reopen outcomes; and
- explicit failure analysis by category and severity.

Proof	What V1 must demonstrate
Technical POC	One mTAP slice completes intake, versioned knowledge compilation, retrieval, Evidence Packet construction, typed decision, verification, policy, SON review, and audit on re-played or approved test inputs.
Operational POV	On Fahad's same baseline cases, the new path retrieves the expected evidence more robustly, avoids unsupported resolution, handles corrections safely, and lets a reviewer reach a defensible decision with less or more useful work than the prior workflow.
Reuse proof	KIOSK can be added as a different support slice through MOHID package content, policies, and evaluations without rewriting the runtime lifecycle.

No product-market-fit claim

V1 may demonstrate technical and operational value for MOHID. It does not establish product-market fit, universal enterprise applicability, or safe automation outside the evaluated categories.

5 V1 Scope and Non-Scope

5.1 In Scope

- controlled manual and API support-request intake;
- a deterministic HelpDocs knowledge compiler with raw snapshots, canonical structured JSON, generated Markdown, versioned search projections, structural diff, and controlled local corpus publication;
- persistent dedicated vector retrieval from the start;
- source URLs, headings, versions, content hashes, and ingestion provenance;
- semantic retrieval with metadata filters and an evaluation-gated lexical or hybrid path;
- multi-intent decomposition and Evidence Packet construction;
- typed resolution, dispatch, and routing decisions;

- response drafting with claim-to-source citations and HelpDocs links;
- schema, evidence, procedure, scope, conflict, completeness, and policy verification;
- category-configurable automatic-versus-review dispatch;
- Level 1 / Level 2 or configured queue routing;
- SON queue, case detail, automatic-resolution review, and chronological audit history;
- approve, edit, reject, clarify, escalate, and reroute reviewer actions;
- narrow deterministic tools for response dispatch, ticket prefilling, and routing;
- durable case, run, decision, tool, event, feedback, and reopen persistence;
- historical replay, shadow mode, universal approval, and narrow auto-resolution; and
- a platform-wide and tenant-scoped automation kill switch.

5.2 Explicitly Deferred

- [DEFERRED] universal multi-tenant self-service administration;
- [DEFERRED] arbitrary no-code workflow construction or tenant code execution;
- [DEFERRED] complex organization-chart builders and connector marketplaces;
- [DEFERRED] generalized MCP infrastructure, MCP hosting/discovery/administration, and multiple interchangeable real knowledge connectors;
- [DEFERRED] live runtime HelpDocs querying and direct creation, modification, deprecation, or publication of authoritative HelpDocs content;
- [DEFERRED] employee leaderboards, disciplinary conclusions, or reductive reviewer scores;
- [DEFERRED] broad workforce analytics;
- [DEFERRED] voice support and mobile applications;
- [DEFERRED] billing, licensing, and commercial subscription management;
- [DEFERRED] destructive or privileged autonomous account actions;
- [DEFERRED] Kubernetes or enterprise-scale infrastructure without measured need;
- [DEFERRED] complete replacement of MOHID's ticketing or communication systems; and
- [DEFERRED] second-tenant productization.

6 Users, Roles, Teams, and Permissions

[V1 DESIGN] V1 supports bounded roles and queues without constructing a general organization builder.

Role	Primary responsibilities	V1 permissions
Requester	Supplies support request and clarification; receives response	Create request through enabled channel; provide clarification; submit feedback; reopen through configured path
Level 1 reviewer	Handles routine or prepared cases	View assigned queue; inspect evidence; approve; edit; request clarification; reject; reroute; escalate
Level 2 / specialist	Handles exceptions, complex products, or restricted categories	All Level 1 actions plus specialist resolution, privileged recommendation, and return-to-Level-1 action

Role	Primary responsibilities	V1 permissions
Manager / operations owner	Controls queues, policy rollout, and exception ownership	View all authorized queues; reassign; approve category rollout; pause automation; inspect operational metrics
Knowledge owner	Maintains authoritative documentation	Approve local corpus publication; review knowledge-gap evidence and proposed corrections; external HelpDocs modification or publication remains separately authorized and deferred
System administrator	Maintains identity, integrations, secrets, and environment configuration	Configure adapters, queues, and policies; manage kill switch; no default right to alter case decisions
Auditor / quality reviewer	Samples outcomes and reconstructs decisions	Read-only access to cases, evidence, events, tool receipts, human decisions, and version metadata

6.1 Initial Queue Model

[V1 DESIGN] V1 begins with:

- **No human queue:** case may auto-resolve and remain available for sampling.
- **Central / Level 1:** normal review and clarification.
- **Level 2 / specialist:** complex, unsupported, restricted, or product-specific cases.
- **Manager:** policy exceptions, repeated failure, or manual ownership decision.

Routing policies may later map product or category queues, but V1 stores route type, queue, team, and optional owner as separate fields so organization growth does not require a schema rewrite.

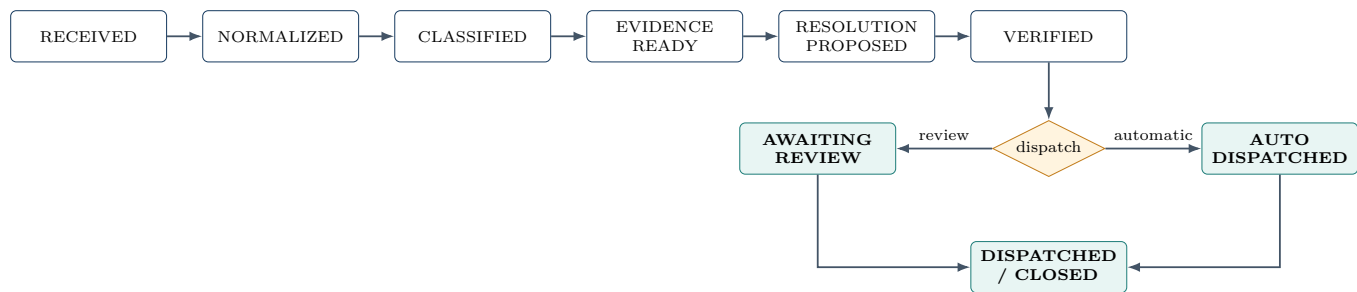
6.2 Permission Invariants

- Every human action requires an authenticated actor and current case version.
- Editing a proposed response does not grant permission to dispatch it.
- Rerouting does not change the resolution decision.
- A reviewer cannot approve a tool action outside their assigned permission scope.
- Authentication to an API or MCP server does not grant a support capability; runtime policy still validates the tenant, application, actor, environment, case/run, requested scope, evidence, approval, and kill-switch state.
- Policy blocks cannot be overridden by a lower-privilege reviewer.
- All overrides require a reason code; free text is optional unless policy requires it.

7 End-to-End Case Lifecycle

7.1 Case and Run Semantics

The **Support Case** is the durable human-facing record. A **Request Run** is one versioned processing attempt triggered by intake, replay, clarification, reopening, reviewer retry, or policy/model/corpus evaluation. Runs never overwrite one another.



7.2 Canonical States

State	Meaning and entry invariant	Owner
RECEIVED	Authenticated or trusted manual intake has been durably recorded with an idempotency key	Intake service
NORMALIZED	Original content is preserved; channel noise, quoted history, and attachments are represented separately	Workflow
CLASSIFIED	One or more intent units and preliminary risk/category labels exist	Workflow
EVIDENCE_READY	A versioned Evidence Packet exists, including insufficiency or conflict when present	Retrieval/evidence
RESOLUTION_PROPOSED	Typed per-intent resolution and bounded customer-facing draft or non-response action exist	Resolution
VERIFIED	Required verification checks completed and produced structured results	Verification
DISPATCH_DECIDED	AUTOMATIC, HUMAN_APPROVAL, or DENIED is recorded with policy version and reasons	Policy
AWAITING_REVIEW	Human action is required and route ownership is resolved	SON queue
APPROVED	Reviewer accepts the prepared resolution and authorized action	Reviewer
EDITED	Reviewer changed the response or structured decision; changed claims require re-verification	Reviewer/workflow
CLARIFICATION_REQUESTED	Required information is missing and a bounded clarification is prepared or sent	Reviewer/workflow
ESCALATED	Case is transferred to a higher-authority queue or specialist	Routing
REJECTED	Request is prohibited, invalid, abusive, duplicate, or outside the supported boundary	Policy/reviewer
REROUTED	Ownership changed without collapsing resolution or dispatch	Reviewer/routing
AUTO_DISPATCHED	All automatic gates passed, persistence succeeded, and the action receipt is recorded	Tool/dispatch
DISPATCHED	Approved response or tool action completed and receipt/reconciliation state is durable	Tool/dispatch
CLOSED	No current action is required; final outcome and timing snapshot exist	Workflow
FEEDBACK_RECEIVED	Post-resolution feedback is attached to the closed case	Feedback
REOPENED	New evidence or customer response creates a new Request Run under the same case	Intake/workflow

7.3 Failure and Recovery States

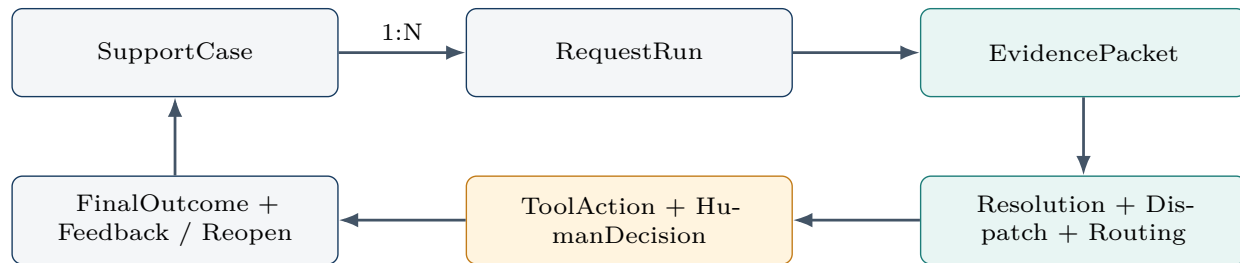
INTAKE_REJECTED	Authentication, size, schema, tenant, or replay validation failed; no downstream side effect.
PROCESSING_FAILED	Normalization, classification, retrieval, generation, or verification failed after bounded retries.
ACTION_FAILED	Tool execution did not produce a valid receipt; reconcile before retry.
DISPATCH_FAILED	Outbound delivery failed or is indeterminate; case remains open and cannot be marked dispatched.
DEAD_LETTERED	Retry budget exhausted; route to operations with full trace.
CANCELLED	Authorized operator stopped a case or run; reason and actor are recorded.

7.4 Transition Rules

- Transitions are compare-and-swap operations on a monotonically increasing case version.
- Any side effect requires durable intent and idempotency key before execution.
- An edited substantive claim returns to verification before dispatch.
- A clarification response creates a new Request Run; it does not mutate the original run.
- A reopened case retains prior final outcome and starts a new run with a reopen reason.
- Failed automatic dispatch never silently degrades into “closed.” It enters a recorded failure state—`ACTION_FAILED` or `DISPATCH_FAILED`—and routes to review.
- Terminal run states are immutable. The case may continue through a later run.

8 Domain Model and Contracts

8.1 Relationship Model



8.2 Required Objects

Object	Required fields	Invariant
SupportCase	case_id, tenant_id, application_id, channel, external_thread_ref, requester_ref, state, version, active_run_id, category, risk flags, route, created/updated/closed timestamps	Human-facing unit of truth; never replaced by a model run or message
RequestRun	run_id, case_id, run number, trigger, environment, runtime/application-package/workflow versions, corpus version, model-profile version, policy version, adapter versions, status, timestamps, failure class	Immutable processing attempt and configuration snapshot; later attempts create new rows
SourceDocument	document_id, canonical URL, title, product/category, language, owner, status, effective dates, current version	Identity remains stable across content versions

Object	Required fields	Invariant
SourceDocument Version	version_id, document_id, content hash, raw snapshot ref, canonical block tree, generated Markdown ref, structural-diff ref, links, fetched/published timestamps, parser version	Exact source used by a run is reconstructable; generated artifacts cannot replace canonical blocks
DocumentChunk	chunk_id, document version, ordinal, heading path, block types, text, locator, chunk hash, embedding version	Chunk never loses its source version or structural locator
CorpusVersion	corpus_version, document-version set, parser/chunker/embedding versions, validation report, structural-diff ref, approval actor/time, publication status	Published corpus is immutable and only APPROVED versions may serve automatic evidence
EvidencePacket	packet_id, run_id, corpus version, intent evidence, source spans, conflicts, missing requirements, coverage, retrieval trace	Contains supporting and disqualifying evidence; not merely top-k chunks
ResolutionDecision	decision_id, run_id, top-level state, per-intent states, allowed response scope, unresolved questions, proposed response, rationale codes	Uses RESOLVE, CLARIFY, ESCALATE, REJECT, or NO_ACTION only
VerificationResult	verification_id, run_id, check results, claims, support mappings, procedure checks, conflicts, unsupported claims, pass status, verifier version	Failed or unavailable verification cannot permit automatic dispatch
PolicyDecision	policy_decision_id, run_id, policy version, evaluated rules, results, reason codes, timestamp	Deterministic and reproducible from recorded inputs
DispatchDecision	decision_id, run_id, outcome, reason codes, policy decision, reviewer requirement	Uses AUTOMATIC, HUMAN_APPROVAL, or DENIED only
RoutingDecision	decision_id, run_id, route type, queue/team/owner, reason codes, service target, policy version	Ownership remains independent of resolution and dispatch
ToolAction	action_id, case/run, tool type/version, typed input, authorization, policy decision, dry-run result, idempotency key, status, attempts, receipt	Model proposes intent; deterministic code executes
HumanDecision	decision_id, case/run, actor, action, expected case version, reason code, structured changes, text diff, timestamp	Every consequential human change is attributable and version-checked
AuditEvent	event_id, case/run, sequence, actor type/ref, event type, payload, payload hash, occurred/recorded timestamps	Append-only and ordered within a case
FinalOutcome	outcome_id, case/run, disposition, dispatch status, external refs, close reason, timing summary, quality flags	Created only after terminal state and receipt reconciliation
FeedbackEvent	feedback_id, case, source, rating/type, comment, received timestamp	Adds outcome evidence without rewriting history
ReopenEvent	reopen_id, case, prior outcome, reason, new message/event ref, timestamp, new run id	Preserves prior closure and starts a new run

8.3 Canonical Decision Bundle

```
{
  "case_id": "case...",
  "run_id": "run...",
  "evidence_packet_id": "evp...",
  "resolution": {
    "state": "RESOLVE | CLARIFY | ESCALATE | REJECT | NO_ACTION",
    "per_intent": [
      {
        "intent_id": "intent_1",
        "state": "RESOLVE",
        "evidence_span_ids": ["span_1", "span_2"],
        "unresolved_requirements": []
      }
    ],
    "allowed_response_scope": ["documented_procedure"],
    "proposed_response": "...",
    "rationale_codes": ["EVIDENCE_SUFFICIENT"]
  },
  "dispatch": {
    "outcome": "AUTOMATIC | HUMAN_APPROVAL | DENIED",
    "policy_version": "mohid-policy-...",
    "reason_codes": ["CATEGORY_REVIEW_ONLY"]
  },
  "routing": {
    "route_type": "NONE | CENTRAL | LEVEL_1 | LEVEL_2 | SPECIALIST | MANAGER",
    "queue_id": "queue...",
    "team_id": null,
    "owner_id": null,
    "reason_codes": ["DEFAULT_LEVEL_1"]
  }
}
```

8.4 Contract Rules

- Identifiers are opaque and immutable.
- Every artifact includes tenant, case, run, and version context where applicable.
- Enumerations are validated at boundaries; unknown values fail closed.
- Customer-facing text never serves as the only representation of a decision.
- Structured reason codes support reporting; optional notes preserve nuance.
- Provider response bodies are stored only through redacted, access-controlled receipts.
- Schemas use explicit version fields and backward-compatible migrations.
- Every RequestRun pins one validated compatibility tuple; configuration changes affect only subsequent runs.

9 HelpDocs Ingestion and Knowledge Lifecycle

9.1 Production Knowledge Model

[**CONFIRMED**] The HelpDocs website is authoritative. [**V1 DESIGN**] V1 stores four linked representations:

1. **Raw snapshot:** fetched HTML and response metadata for reproducibility.
2. **Canonical document:** structured JSON preserving headings, paragraphs, lists, procedures, warnings, tables, labels, links, and source metadata.
3. **Generated review artifact:** deterministic Markdown rendered from the canonical block tree for side-by-side inspection, Git-style diffing, and corpus approval.
4. **Search projections:** versioned chunks and embeddings in a persistent vector database, with optional lexical fields or index.

The crawler or fetcher is an acquisition component. It is not the retrieval system. Markdown is a human-readable derivative, not the canonical system record and not an intermediate that may reinterpret the source before chunking.

HTML → raw snapshot → canonical block tree

canonical block tree ⇒ generated Markdown + search projections

9.2 Acquisition Boundary and Initial Adapter

An external HelpDocs API, export, HTML site, or other supported interface would supply the underlying source capability. An MCP server, if one later exists or is justified, would be only an optional AI-facing wrapper around that capability. MOHID V1 calls a semantic **KnowledgeSourceProvider**; it does not expose API or MCP payloads to canonical documents, Evidence Packets, decisions, cases, or policy.

Package A begins with a fake source provider and an approved HTML or fixture-backed HelpDocs adapter. Official HelpDocs API or export access is unconfirmed and is not a prerequisite for the mTAP proof. If MOHID later enables an official interface, a direct REST/SDK adapter or MCP-backed adapter may replace acquisition beneath the same contract. Neither option bypasses raw snapshotting, canonicalization, structural diff, validation, approval, or corpus publication.

9.3 Ingestion Pipeline

1. discover only approved HelpDocs pages through the configured KnowledgeSourceProvider, initially using a sitemap, manifest, allowlisted crawl, or approved fixtures;
2. fetch with bounded concurrency, timeout, retry, user-agent, and response validation;
3. store raw content, canonical URL, fetch headers, timestamp, and checksum;
4. remove global navigation, footers, cookie banners, and repeated boilerplate;
5. parse semantic blocks while preserving heading hierarchy and ordered procedures;
6. render Markdown deterministically from the canonical blocks;
7. validate minimum content, URL uniqueness, language, source status, structural integrity, and source-to-canonical fidelity;
8. compute document and block hashes;
9. compare against the last published version to classify new, changed, unchanged, removed, redirected, or deprecated pages;
10. produce a structural diff that distinguishes content, hierarchy, procedure, warning, link, and metadata changes;
11. generate structure-aware chunks with deterministic locators;
12. create embeddings through the EmbeddingProvider adapter;
13. upsert chunks to the VectorStore and persist metadata in PostgreSQL;
14. publish a new immutable corpus version only after validation succeeds and the designated knowledge owner approves the structural diff for the target environment; and
15. emit audit events and a machine-readable sync report.

9.4 Canonical Document Contract

```
{
  "document_id": "doc_...",
  "version_id": "docv_...",
  "canonical_url": "https://mohid.helpdocsite.com/article/...",
  "title": "...",
  "product": "mohid",
  "category": "kiosk | donation | registration | ...",
  "status": "APPROVED | DEPRECATED | REMOVED | DRAFT",
```

```
"content_hash": "sha256:...",
"parser_version": "helpdocs-parser-1",
"fetched_at": "...",
"heading_tree": [..., "..."],
"blocks": [
  {
    "block_id": "blk...",
    "type": "paragraph | ordered_steps | warning | table | link",
    "heading_path": [..., "..."],
    "ordinal": 3,
    "text": "...",
    "links": [{"label": "...", "url": "..."}],
    "source_locator": "article > section:nth-of-type(2)"
  }
]
```

9.5 Generated Markdown and Parser Discipline

The canonical parser and Markdown scaffolder are deterministic application code. A model may assist an engineer in diagnosing an unknown layout, but it may not silently rewrite authoritative text, reorder procedures, invent headings, or publish a corpus version.

Five representative mTAP pages form the initial golden parser set: a simple article, a multi-step procedure, a nested-heading page, a page with warnings or prerequisites, and a page with links, media, or unusual formatting. The generated Markdown is reviewed beside the original page, while the canonical JSON retains block type, ordinal, hierarchy, link target, and source locator needed for exact replay.

9.6 Chunking Rules

[V1 DESIGN] The default chunker is structure-aware:

- procedures remain intact when they fit the configured token budget;
- headings are repeated as metadata, not blindly duplicated into every body;
- warnings and prerequisites remain attached to the procedure they qualify;
- long sections split at semantic block boundaries with limited overlap;
- tables retain header context;
- each chunk records document version, heading path, block range, and canonical URL; and
- chunk strategy and embedding version are recorded for replay.

Character or token windows may be a fallback for unusually long blocks, but fixed-size flattening is not the production default.

The initial benchmark compares heading-based sections and complete procedure blocks. Parent-child retrieval, larger parent-context expansion, lexical fusion, and reranking are candidates only when the mTAP replay set shows a measurable gain. No fixed parent/child token size is an architectural requirement.

9.7 Synchronization and Deletion

Change	Required behavior
New page	Create document identity and version; validate; chunk; embed; include only in next published corpus
Changed page	Create new version; retain old version; produce structural diff; replace current search projection atomically only after approved corpus publish
Unchanged page	Record observation without re-embedding
Removed page	Mark removed; delete current chunks deterministically; retain historical version for run reconstruction

Change	Required behavior
Deprecated page	Exclude from automatic dispatch; retrieval may expose it only for controlled review if policy permits
Redirected page	Preserve old URL relationship and publish canonical target
Failed fetch	Do not delete last good version; mark sync incomplete and block corpus publish if coverage policy fails

9.8 Initial Implementation Choice

[V1 DESIGN] V1 uses PostgreSQL for document metadata, corpus publication state, cases, and audit records. Pgvector is the proposed first VectorStore implementation because the same data model can run in MOHID's Google Cloud environment or on MOHID-managed PostgreSQL infrastructure. Raw snapshots use company-approved object storage behind ObjectStore. The initial fake and HTML/fixture source adapters must pass one KnowledgeSourceProvider contract suite. Gemini embeddings are the first benchmark candidate through MOHID-owned Google Cloud access; the selected model must pass the same EmbeddingProvider contract and retrieval tests as the local fake.

9.9 Ingestion Acceptance Criteria

- Every published chunk resolves to one exact canonical URL and document version.
- Re-running an unchanged corpus produces no duplicate vectors or document versions.
- Removed or deprecated content cannot appear in automatic-dispatch Evidence Packets.
- A changed page can be re-indexed without leaving old current-version chunks searchable.
- Headings, lists, ordered steps, warnings, and links survive canonicalization on the test corpus.
- Generated Markdown and search chunks derive from the same canonical blocks and cannot disagree on instruction order or source identity.
- Five golden mTAP pages preserve every material instruction, warning, ordering relationship, link, and heading in side-by-side review.
- A changed production corpus cannot become active without a reviewable structural diff and recorded publication approval.
- A failed fetch cannot silently erase previously published knowledge.
- Fake and HTML/fixture-backed source adapters obey the same discovery, fetch, checkpoint, metadata, and failure semantics; transport-specific payloads do not enter canonical or evidence contracts.
- Sync reports list counts, errors, corpus version, parser version, embedding version, and hashes.

10 Retrieval and Evidence Packet Design

10.1 Retrieval Sequence

[V1 DESIGN] The baseline sequence is:

normalize → decompose intents → construct queries → semantic + metadata retrieval → optional lexical fusion → rerank if justified → qualify evidence → Evidence Packet

The first implementation may begin with semantic retrieval plus metadata filters. Keyword or hybrid retrieval and reranking are added only when the replay benchmark shows a meaningful gain. The contract supports them from the start.

Normalization removes transport noise, quoted history, signatures, greetings, and non-semantic wrappers while preserving product terms, negation, identifiers, and the untouched original request. The mTAP clean/noisy pair is a required paired test: the expected evidence and disposition should remain stable even when raw ranking scores change.

10.2 Live Knowledge Query Boundary (Deferred)

The mTAP proof retrieves from the approved local corpus and does not require a live HelpDocs call. A protocol-neutral `KnowledgeQueryProvider` seam may later support search, exact-document retrieval, and source-version inspection through direct REST, SDK, database, or MCP adapters.

Any future live result used in a support decision must first be normalized into the existing source-document and evidence-span contracts; the raw result is snapshotted, hashed, timestamped, assigned stable span identifiers, bound to the Support Case and Request Run, and retained for replay. A changing API or MCP response may not serve as unrecorded transient evidence. Connector authentication also does not establish source authority or permit automatic dispatch; those remain runtime policy decisions.

10.3 Intent Decomposition

Each request receives one or more intent units. An intent includes normalized text, product/category hypotheses, required entities, candidate query variants, and risk flags. The original request remains unchanged.

Per-intent processing prevents a compound email from being declared resolved because one sub-question was answered. Partial resolution is allowed only when policy permits it and the response clearly distinguishes resolved and unresolved intents.

10.4 Corrections, Replies, and Reopening

A reply such as “this is not the right answer” is not a new knowledge query by itself. When thread identity or a signed case reference is available, it appends feedback to the existing Support Case and creates a new Request Run whose input includes the prior request, prior response, cited evidence, and correction. If the case cannot be resolved safely, the system asks a bounded clarification or routes the case for human review rather than searching the correction sentence as an unrelated request.

10.5 Evidence Qualification

Candidate chunks are qualified using:

- approved source and current document status;
- corpus, parser, chunker, and embedding version;
- product/category compatibility;
- source URL and exact structural locator;
- procedure completeness and prerequisite coverage;
- required field and entity coverage;
- freshness and deprecation state;
- conflict or contradiction detection;
- cross-intent duplication and missing evidence; and
- explicit insufficiency when no defensible packet exists.

Raw similarity score is diagnostic evidence, not factual confidence and not dispatch authority.

10.6 Evidence Packet Contract

```
{
  "packet_id": "evp...",
  "run_id": "run...",
  "corpus_version": "mohid-corpus...",
  "intents": [
    {
      "intent_id": "intent_1",
      "normalized_intent": "...",
      "coverage": "SUFFICIENT | PARTIAL | NONE | CONFLICTED",
      "source_spans": [
        {
```

```

    "span_id": "span_1",
    "document_version_id": "docv_...",
    "chunk_id": "chunk_...",
    "heading_path": ["...", "..."],
    "quote": "...",
    "canonical_url": "..."
  }
],
"missing_requirements": [],
"conflicts": []
}
],
"retrieval_trace": {
  "queries": ["..."],
  "candidate_ids": ["..."],
  "ranking_version": "retrieval-...",
  "embedding_version": "...",
  "latency_ms": 0
}
}

```

10.7 Conflict and Insufficiency Behavior

- Substantive unresolved conflict forces HUMAN_APPROVAL or DENIED.
- Missing required input produces CLARIFY when a bounded question can resolve it.
- Missing authority, account access, or specialist judgment produces ESCALATE.
- No approved evidence prevents RESOLVE.
- Deprecated evidence may inform a reviewer but cannot justify automatic dispatch.

10.8 Retrieval Evaluation

The replay harness must report:

- expected-source recall at configured k;
- evidence coverage by intent;
- citation locator correctness;
- conflict detection and insufficiency accuracy;
- source-status violations;
- active-context recall: required retrieved sources that actually enter the Evidence Packet rather than a display-only list;
- paired clean/noisy intent stability, including rank and coverage deltas;
- correction/reply handling against the correct prior Support Case;
- latency and provider cost; and
- failure slices by messy phrasing, multi-intent, category, and procedure type.

11 Resolution, Verification, Policy, Dispatch, and Routing

11.1 Resolution

Resolution answers *what should happen*:

RESOLVE	Approved evidence and authority support an answer or bounded action.
CLARIFY	One or more explicit missing facts prevent safe resolution and can be requested.
ESCALATE	Higher authority, account access, exception handling, or specialist judgment is required.
REJECT	The request is prohibited, invalid, abusive, duplicate, or outside the supported boundary.
NO_ACTION	No outbound action is required, such as a notification or duplicate event.

The resolution engine produces per-intent states before final customer-facing drafting.

11.2 Verification

Check	Question	Automatic failure behavior
Schema	Are required fields, enum combinations, references, and versions valid?	DENIED
Source status	Is every cited source approved and current for the corpus?	DENIED
Claim support	Does each substantive claim map to one or more evidence spans?	HUMAN_APPROVAL or DENIED
Procedure completeness	Are prerequisites, ordered steps, warnings, and completion conditions present?	HUMAN_APPROVAL
Terminology	Do product names, labels, and links match approved sources?	HUMAN_APPROVAL
Multi-intent coverage	Does every intent have an explicit, non-misleading disposition?	HUMAN_APPROVAL
Conflict	Does unresolved evidence contradict the proposed answer or action?	DENIED
Scope and sensitivity	Is the request inside the enabled category and allowed data boundary?	HUMAN_APPROVAL or DENIED
Tool readiness	Are required inputs, authorization, dry run, and idempotency available?	DENIED
Persistence	Are the packet, decisions, and intended action durable before execution?	DENIED

Model-based verification may assist with claim extraction or semantic comparison. Deterministic checks control schemas, versions, permissions, policy, idempotency, and execution.

11.3 Dispatch

Dispatch answers *may the platform execute now*:

- **AUTOMATIC**: all category, evidence, verification, permission, tool, persistence, and rollout gates pass.
- **HUMAN_APPROVAL**: a reviewer must approve, edit, clarify, reject, reroute, or escalate.
- **DENIED**: execution is prohibited or prerequisites cannot be satisfied.

11.4 Routing

Routing answers *who owns the case or exception*. It may choose no queue, central/Level 1, Level 2, specialist, manager, or a configured product/function queue. Routing stores reason codes and expected service target; it does not alter the resolution.

11.5 Automatic-Dispatch Gate

A case is eligible for AUTOMATIC only when:

1. intake is authentic, non-duplicate, and inside size/rate limits;
2. the category is explicitly enabled for the current rollout stage;

3. every intent has a supported disposition;
4. the Evidence Packet is sufficient and has no substantive unresolved conflict;
5. every substantive claim and procedural requirement passes verification;
6. no restricted data, policy exception, or privileged action is involved;
7. the requested tool and exact action are allowlisted;
8. all required fields and authorization context exist;
9. persistence succeeds before the side effect;
10. the tenant and platform kill switches permit automation; and
11. current evaluation and production health remain inside release thresholds.

Model confidence is never sufficient

A model confidence score, retrieval score, or fluent response cannot by itself satisfy evidence, policy, authorization, or rollout gates.

11.6 Policy Representation

[V1 DESIGN] V1 begins with versioned application policy expressed as typed configuration and deterministic functions. OPA or another policy engine remains an extension point, not an initial dependency.

Minimum policy inputs include category, product, source status, evidence coverage, conflict state, risk class, channel, action type, required fields, reviewer role, rollout phase, sample rate, and kill-switch state.

12 Tool Execution and External Integrations

12.1 Capability and Authorization Separation

MOHID keeps four capability classes distinct: batch knowledge synchronization, deferred live knowledge querying, deferred knowledge-base writing, and operational business actions. The first mTAP slice implements batch synchronization plus only the bounded operational actions listed below. It does not grant one broad “MOHID connector” authority, and knowledge credentials cannot imply ticket, account, receipt, routing, or publication permission.

Models may propose a typed tool intent. Deterministic runtime code validates the tenant, application package, actor, environment, source authority, case and run, requested capability, granted scope, category and risk, evidence, required human approval, policy version, expected resource version, rollout stage, idempotency, and kill-switch state. Only then may ToolExecutor receive an authorized request. API credentials or MCP authorization establish connector access, not permission to perform the support action.

12.2 V1 Tool Catalog

Tool	Purpose	Initial execution rule
create_response_draft	Persist customer-facing draft and citations	No external side effect; allowed after evidence packet
send_approved_response	Send response through configured outbound channel	Human approval until narrow auto phase
create_ticket_prefill	Produce non-privileged ticket payload or form	Dry run first; creation requires policy and configured endpoint
route_case	Assign queue/team/owner	Deterministic, auditable; may execute automatically
escalate_case	Create escalation payload and route to higher authority	Must validate target; never silently succeed without receipt
send_clarification	Request missing information	Human approval during initial pilot

Tool	Purpose	Initial execution rule
send_helpdocs_link	Send a bounded approved link and explanatory text	Eligible for later category-specific automation

12.3 ToolAction Contract

```
{
  "action_id": "act...",
  "case_id": "case...",
  "run_id": "run...",
  "tool_type": "send_approved_response",
  "tool_version": "1",
  "input": {"to_ref": "requester...", "message_id": "msg..."},
  "authorization": {
    "actor_type": "HUMAN | SYSTEM",
    "actor_ref": "...",
    "capability": "message.send",
    "grant_id": "grant..."
  },
  "policy_decision_id": "pol...",
  "dry_run": {"status": "PASSED", "validated_at": "..."},
  "idempotency_key": "case...:run...:send_response:1",
  "status": "PROPOSED | AUTHORIZED | EXECUTING | SUCCEEDED | FAILED | UNKNOWN",
  "attempts": 0,
  "receipt": null
}
```

12.4 Execution Invariants

- Every tool has versioned input and output schemas.
- Runtime authorization and policy are evaluated before executor handoff; ToolExecutor rejects a missing, expired, mismatched, or invalid authorization envelope.
- Dry-run validation is required where an integration supports it.
- Side effects use durable idempotency keys and bounded retries.
- Consequential writes use least-privilege credentials, explicit allowed operations, schema validation, and expected-version protection where the provider supports it.
- Timeout or connection loss can produce UNKNOWN, which requires receipt reconciliation before retry.
- Success requires a validated provider or destination receipt.
- Failure creates an explicit case state and audit event.
- The model never directly receives provider credentials or arbitrary network access.

12.5 Email Integration Sequence

1. **[V1 DESIGN] Contract-first local flow:** exercise inbound, draft, approval, and outbound contracts with fakes and no network side effects.
2. **[V1 DESIGN] Office 365 shadow flow:** after MOHID authorizes the shared mailbox, ingest copied or forwarded support requests through Microsoft Graph or the approved forwarding mechanism without sending a customer response.
3. **[V1 DESIGN] Mandrill test-recipient flow:** send only approved test messages through MOHID's transactional email path and persist provider receipts.
4. **[V1 DESIGN] Universal-review pilot:** use the same Office 365 inbound and Mandrill outbound adapters with human approval on every customer-facing response.
5. **[DEFERRED]** Direct replacement of MOHID's existing ticketing or mail operations.

The earlier Mailgun path supplies fixtures only. It is not an alternate first-implementation channel.

12.6 Integration Preconditions

[OPEN] Microsoft Graph tenant, mailbox, consent, scopes, Mandrill credentials, sender identity, and escalation destination must be refreshed before live shadow or test-recipient work. The MOHID Google Cloud project, Gemini/Vertex access, hosting target, secrets, networking, cost ownership, and operations owner must be confirmed before the integrated environment is created.

13 SON V1 Product Requirements

13.1 Product Surface

SON is required in V1. It is the human control plane, not an optional analytics dashboard.

13.2 Screen 1: Case Queue

Primary purpose	Show cases requiring intervention and samples of automatic resolutions.
Required columns	Case ID, age, state, category, resolution, dispatch, route, evidence status, risk flags, current owner, last event.
Filters	My queue, unassigned, Level 1, Level 2, state, category, route, age, policy block, verification failure, reopened, auto-resolved sample.
Sort	Oldest first, newest first, risk, evidence-ready time, current queue wait.
Bulk actions	None for consequential decisions in V1; bulk assignment may be added with audit.
Empty/error states	Explain whether no cases match, data is unavailable, or access is denied.

13.3 Screen 2: Case Detail

The case detail must include:

1. **Header:** state, category, dispatch, route, owner, age, risk, and kill-switch status.
2. **Original request:** preserved content, subject, channel, thread context, and attachments.
3. **Proposed outcome:** per-intent resolution, proposed response, and unresolved questions.
4. **Evidence:** source titles, exact spans, URLs, headings, versions, conflicts, and missing requirements.
5. **Verification and policy:** pass/fail checks, reason codes, dispatch outcome, and blocked conditions.
6. **Reviewer action bar:** approve, edit, reject, request clarification, reroute, and escalate.
7. **Case timeline:** ordered AI, human, tool, routing, feedback, and reopen events.
8. **Technical drawer:** request runs, model/embedding/retrieval/policy versions, latency, and tool receipts for authorized roles.

13.4 Reviewer Action Requirements

Action	Required input	System behavior
Approve	Expected case version; optional note	Authorize prepared response/action; record actor and timestamp
Edit	Edited response or structured fields; reason code when substantive	Save diff; re-run claim/procedure verification before dispatch
Reject	Reason code; optional note	Block outbound action; transition to rejected or route as configured
Clarify	Missing fields or clarification text	Persist clarification decision; send only after dispatch policy passes
Reroute	New queue/team/owner; reason code	Change ownership without changing resolution

Action	Required input	System behavior
Escalate	Target and reason code; optional summary	Create escalation action and require destination receipt

13.5 Screen 3: Automatic-Resolution Review

This view uses the same case-detail object and supports post-dispatch sampling, policy failures, customer negatives, and reopened cases. Reviewers can mark correct, minor issue, major issue, unsafe, knowledge gap, policy gap, routing issue, or tool failure. Corrections do not erase the dispatched record.

13.6 Meaningful Review Requirements

- Evidence and unresolved risk appear before the approval control.
- Source links open the exact authoritative page.
- Edited text shows a diff against the proposal.
- Disabled actions explain which permission or policy blocks them.
- Concurrent updates produce a visible conflict rather than last-write-wins loss.
- Queue and case views do not infer motive or display reductive employee scores.
- Keyboard navigation, readable contrast, clear focus, and screen-reader labels are required.

14 Persistence, Audit, and Timing Model

14.1 Persistence Boundaries

[V1 DESIGN] PostgreSQL is the source of truth for cases, runs, decisions, events, users, teams, policies, tool actions, and outcomes. Vector search is a derived search projection. Object storage holds raw source snapshots and large artifacts. No derived index may become the only copy of a business record.

14.2 Initial PostgreSQL Tables

Table family	Tables
Identity and organization	tenants, users, roles, user_roles, teams, queues, queue_memberships
Case lifecycle	support_cases, messages, attachments, request_runs, state_transitions
Knowledge	source_documents, source_document_versions, document_blocks, document_chunks, corpus_versions, ingestion_runs, corpus_publication_approvals
Evidence and decisions	intent_units, evidence_packets, evidence_spans, evidence_conflicts, resolution_decisions, verification_results, policy_decisions, dispatch_decisions, routing_decisions
Actions and outcomes	tool_actions, tool_attempts, human_decisions, final_outcomes, feedback_events, reopen_events
Governance	audit_events, policy_versions, automation_controls, evaluation_datasets, evaluation_cases, evaluation_runs, metric_results

14.3 AuditEvent Contract

```
{
  "event_id": "evt...",
  "tenant_id": "mohid",
  "application_id": "mohid-support",
  "case_id": "case...",
  "run_id": "run...",
  "sequence": 42,
  "event_type": "HUMAN_EDITED_RESPONSE",
  "actor": {"type": "HUMAN", "ref": "user..."},
}
```

```

"payload": {"decision_id": "hdec...", "reason_code": "PROCEDURE_FIX"},
"payload_hash": "sha256:...",
"occurred_at": "...",
"recorded_at": "..."
}

```

Audit events are append-only. Corrections produce new events. Access, retention, and redaction rules may hide content from a viewer but must not fabricate a different history.

14.4 Timing Intervals

Metric	Definition	Required events
First-response time	Intake to first customer-facing response or clarification	received, dispatched
Total resolution time	Intake to closed outcome, excluding later reopen run	received, closed
Evidence-ready time	Intake to published Evidence Packet	received, evidence_ready
Queue wait time	Time assigned to a review queue before reviewer opens/claims	routed, claimed/opened
Active reviewer time	Measured active case interaction, bounded by explicit UI events and inactivity policy	opened, action events, closed view
Customer wait time	Time waiting for requester clarification or response	clarification sent, reply received
External-dependency time	Time waiting for provider, specialist, or tool result	wait started/completed
Tool execution time	Authorized execution start to validated receipt	executing, succeeded/failed

These are observable intervals. They do not prove employee motive, effort, or intent.

14.5 Consistency and Recovery

- Case updates use transactions and optimistic concurrency.
- Side effects use a transactional outbox or equivalent durable execution queue.
- Events and state changes commit atomically where possible.
- Backups and restoration are tested before limited production.
- Replay never triggers external actions unless an explicit execution mode and authorization permit it.
- Database migrations are versioned and applied through CI/CD.

15 Security, Privacy, and Tenant Boundaries

15.1 Security Objectives

V1 must protect the confidentiality of requester and tenant data, the integrity of authoritative knowledge, the authenticity of intake, the authorization of actions, the completeness of audit records, and the recoverability of case processing.

Threat	Failure	Required V1 control
Forged or replayed intake	Attacker creates cases or outbound messages	Signed webhook where used, OAuth for Graph, idempotency, timestamp/nonce validation, rate limits
Prompt injection	Request or document text changes system policy or invokes tools	Treat content as data, preserve instruction hierarchy, tool allowlists, typed plans, deterministic authorization

Threat	Failure	Required V1 control
Knowledge poisoning	Unauthorized or stale document enters automatic evidence	Approved-source allowlist, version status, content hashes, publish gate, knowledge-owner role
Cross-tenant leakage	Wrong organization data appears in retrieval or case views	Tenant ID at intake, database filters, row-level controls where practical, vector namespace/filter, negative tests
Excessive agency	Model performs unapproved side effect	No direct credentials, runtime policy authorization before ToolExecutor, dry run, idempotency, receipts
Secret leakage	Credentials enter prompts, logs, or client code	Runtime secret manager, redaction, server-only adapters, access review
HTML or content injection	Unsafe content renders in SON or email	Output escaping, safe Markdown subset, link validation, attachment policy
Audit tampering	Decision history is deleted or rewritten	Append-only events, hashes, restricted mutation, backup, reconciliation
Provider outage or rate limit	Silent loss, duplicate action, or unsafe fallback	Timeouts, bounded retries, circuit breaker, UNKNOWN state, receipt reconciliation, review queue
Unbounded cost or denial of service	Excessive model, retrieval, or tool usage	Per-run budgets, size limits, quotas, timeouts, queue backpressure

15.2 Tenant and Data Isolation

Even though MOHID is the only V1 tenant, every request and record carries `tenant_id`. This prevents accidental coupling and makes isolation testable without building broad self-service tenancy.

- PostgreSQL queries include tenant scope; database row-level controls are defense in depth where available.
- Vector records use a MOHID namespace or mandatory tenant metadata filter.
- Object-store paths and encryption context are tenant-scoped.
- Provider credentials are environment-scoped and never stored in tenant configuration documents.
- Evaluation exports redact requester identifiers unless explicitly required and authorized.

15.3 Privacy and Retention

[OPEN] Retention, redaction, data residency, and requester-identity policy require MOHID approval before live data enters production. Until then:

- use synthetic, redacted, or explicitly approved historical cases;
- store the minimum requester reference needed for threading and response;
- keep raw provider payloads out of general logs;
- restrict individual-level review metrics to authorized roles; and
- expose facts and timing, never inferred motive or disciplinary conclusions.

15.4 Security Release Gates

Live shadow mode is blocked until authenticated intake, role checks, tenant isolation, secret handling, output escaping, and audit completeness pass. External dispatch is additionally blocked until idempotency, receipt reconciliation, kill switch, backup, and rollback are verified.

16 Evaluation Dataset and Metrics

16.1 Dataset Construction

[V1 DESIGN] Development begins with a ten-case redacted bootstrap regression set, not a requirement to adjudicate the full release benchmark before writing code. The initial cases are:

1. the clean mTAP setup query from the Render logs;
2. the same mTAP intent wrapped in a greeting and realistic email phrasing;
3. a combined mTAP setup and hardware-installation request;
4. an mTAP hub-configuration request;
5. an unsupported or insufficiently documented mTAP request;
6. a correction after an incorrect answer;
7. KIOSK ticket-sale capability;
8. KIOSK upgrade assistance;
9. donation-receipt procedure; and
10. program-registration trouble requiring clarification or escalation.

Gemini quota, model timeout, Mailgun/Mandrill delivery, duplicate intake, stale source, and partial-fetch failures are fault-injection variants applied to suitable cases rather than substitutes for business labels.

Before live shadow mode, the benchmark expands to 50 adjudicated cases:

- 15 straightforward documented requests;
- 15 messy, ambiguous, or multi-intent requests;
- 10 unsupported, account-specific, restricted, or escalation cases; and
- 10 adversarial, stale, conflicting, duplicate, malformed, or provider-failure variants.

Each case records the raw or faithfully normalized request, expected intents, required and prohibited sources, expected resolution, dispatch eligibility, route, response properties, severity, corpus version, adjudicator, and notes. A single exact answer string is not required.

16.2 Dataset Splits and Governance

- **Development set:** visible during implementation.
- **Regression set:** fixed cases run on every release candidate.
- **Holdout set:** not used for prompt or retrieval tuning.
- **Production failure set:** new verified failures appended after adjudication.

Label changes require a reason, adjudicator, timestamp, and dataset version. Disputed cases remain excluded from automatic-release thresholds until resolved.

16.3 Required Metrics

Metric	Definition	Primary gate
Relevant-source recall	Expected source appears in candidate set	Retrieval
Active-context recall	Expected retrieved source enters the Evidence Packet rather than only a display or “helpful” list	Evidence

Metric	Definition	Primary gate
Paired-phrasing stability	Clean and realistic noisy forms preserve expected intent, evidence, and disposition within approved variance	Normalization/retrieval
Citation correctness	Cited URL and locator support the attributed statement	Evidence
Evidence coverage	Required evidence present for each intent	Evidence
Conflict detection	Substantive source conflict is surfaced	Evidence/policy
Unsupported-claim rate	Substantive claims without approved evidence	Verification/dispatch
Disposition accuracy	Correct RESOLVE/CLARIFY/ESCALATE/REJECT/NO_ACTION	Resolution
Clarification appropriateness	Clarification is necessary, bounded, and asks for missing data	Resolution
Routing accuracy	Correct queue or no-queue decision	Routing
False auto-resolution rate	Cases automatically resolved when they should not be	Automatic release
Tool success rate	Authorized actions with validated receipts	Tool reliability
Duplicate-action rate	Repeated external side effects for one intended action	Idempotency
Failure recovery	Failed/unknown actions reconciled without unsafe repeat	Reliability
Median review time	Active reviewer time from open to decision	Reviewer leverage
Approval without edit	Review cases accepted as proposed	Reviewer leverage
Minor/major edit rate	Structured edit severity distribution	Quality
Override reason distribution	Why humans disagree with system outputs	Failure analysis
Reopen rate	Closed cases reopened within policy window	Outcome
Correction handling	Negative feedback or correction creates the correct case-aware retry, clarification, or escalation	Lifecycle
Customer feedback	Positive, negative, or unresolved response	Outcome
Model-provider availability	Quota, timeout, and provider failure measured separately from response correctness	Provider reliability
Channel delivery success	Authorized sends with reconciled delivery receipts, separate from answer quality	Channel reliability
First-response time	Intake to first outbound response/clarification	Operations
Total resolution time	Intake to closed outcome	Operations
Safe automation rate	Cases automatically resolved after all gates	Outcome, not optimization target

16.4 Provisional Engineering Gates

These are conservative starting gates, not final client acceptance thresholds:

- zero Critical failures in the release dataset;
- zero unsupported substantive claims among cases proposed for automatic dispatch;
- zero duplicate tool actions in idempotency and retry tests;
- 100% reconstruction of sampled cases from persisted evidence, decisions, actions, and events;
- at least 95% relevant-source recall at the configured candidate depth;
- at least 95% citation correctness and routing accuracy;

- at least 90% overall disposition accuracy; and
- zero observed false auto-resolutions in the enabled-category holdout set before Phase 4.

Threshold authority

Exact production thresholds, sample sizes, category eligibility, and acceptable false-auto-resolution ceiling require explicit approval from the product owner and MOHID adjudication authority. Zero observed errors in a small sample does not prove zero risk.

16.5 Failure Severity

Critical	Tenant leak, unauthorized side effect, material unsupported automatic response, audit loss, or kill-switch failure.
High	Materially wrong/incomplete procedure, incorrect escalation, repeated dispatch failure, or security control failure without known exploitation.
Moderate	Unnecessary clarification, over-escalation, weak wording, minor routing error, or missing nonessential context.
Low	Formatting, tone, cosmetic UI, or non-substantive link-display issue.

17 Rollout and Pilot Plan

Phase	Mode	Work	Exit gate
0	Bootstrap corpus and baseline	Compile the first mTAP pages; freeze the ten-case bootstrap set and legacy traces; record expected evidence, outcome, and known failure	Corpus reproducible; initial labels reviewable; no external actions
1	Offline historical replay	Run retrieval, Evidence Packet, decisions, draft, verification, policy, and routing; expand and adjudicate the benchmark to 50 cases before live entry	Provisional offline gates pass; 50-case pre-pilot benchmark approved; Critical failures resolved
2	Shadow mode	Process live or live-like requests without sending; compare against actual human handling	Intake, persistence, routing, and audit reliable; mismatch review completed
3	Universal human approval	Prepare all outbound responses/actions for SON review; measure edits, overrides, timing, and usability	Review workflow accepted; dispatch/tool controls reliable; thresholds approved
4	Narrow automatic resolution	Enable one or two proven low-risk categories; review all or a high sample after dispatch	No stop condition triggered; category metrics stay inside approved gates
5	Controlled expansion	Add categories or tools one at a time through new evaluation and approval	Each change has dataset coverage, release record, sampling, and rollback

17.1 Kill Switch

The platform exposes:

- a platform-wide automatic-dispatch switch;
- a MOHID tenant switch;
- category and tool enablement controls; and
- a “review all” incident mode.

Disabling automatic dispatch must preserve intake, shadow processing, audit, and reviewer queues.

17.2 Stop Conditions

Automatic dispatch stops immediately for any Critical failure, unrecognized policy version, knowledge sync integrity failure, verifier outage, tenant-isolation failure, repeated provider receipt ambiguity, duplicate side effect, kill-switch failure, or approved threshold breach.

17.3 Rollback

Rollback pins the previous application, policy, prompt/model profile, and corpus versions; disables automatic dispatch; reconciles in-flight tool actions; replays affected cases; and records incident and recovery events. A model rollback without corpus and policy version alignment is incomplete.

18 First Implementation Technology Stack and Engineering Method

18.1 Runtime and MOHID Package Boundary

The runtime implements the fixed support lifecycle, persistence, retrieval execution, verification, policy floor, tool authorization, audit, recovery, and kill switches. The MOHID package supplies only versioned behavior and approved deployment-profile references. Package-schema validation and compatibility checks run before any case can execute; tests must prove that MOHID configuration can narrow automation but cannot bypass a mandatory runtime control.

18.2 Company-Aligned V1 Stack

Layer	First implementation	Portability boundary
SON frontend	DeenTek-approved web stack; Next.js, React, and TypeScript if no internal standard is required	Consumes versioned case APIs; no direct database or provider access
Backend/API	Python, FastAPI, Pydantic	Owns domain contracts, authentication boundary, workflow entrypoints, and tool authorization
Workflow	Explicit typed state machine in Python; LangGraph permitted as the implementation library	Workflow contract and replay tests are authoritative; graph checkpoints never replace PostgreSQL business records
Model access	Gemini through MOHID-owned Google Cloud or the approved Gemini endpoint	ModelProvider contains the Google SDK; domain and policy code receive only typed requests and results
Embeddings	Gemini embedding candidate evaluated in the MOHID environment	EmbeddingProvider fixes the contract; model choice is accepted only through benchmark results
Retrieval	PostgreSQL plus pgvector, with HNSW and metadata filters as the proposed first adapter	VectorStore owns search/upsert/delete contracts; a later store must pass the same suite
Application database	PostgreSQL on MOHID-approved infrastructure	Source of truth for cases, decisions, events, users, teams, policies, and evaluation
Object storage	Google Cloud Storage or MOHID-approved server storage	ObjectStore owns versioning, hashes, retention metadata, and retrieval
Knowledge connectors	Fake plus approved HelpDocs HTML/fixture adapter for batch synchronization; no live-query implementation in the first slice	KnowledgeSourceProvider and KnowledgeQueryProvider express semantic capabilities; REST, SDK, export, crawler, or MCP details remain inside adapters
Inbound email	Office 365 shared mailbox via Microsoft Graph or approved forwarding	InboundMessageProvider normalizes messages and hides channel payloads from the core

Layer	First implementation	Portability boundary
Outbound email	Mandrill/Mailchimp Transactional	OutboundMessageProvider and ToolExecutor own validation, idempotency, receipts, and reconciliation
Hosting	Docker on MOHID-managed servers or an approved MOHID Google Cloud target	Container and environment contracts remain portable; deployment scripts target the approved environment only
Testing and CI	Pytest, testcontainers, contract suites, historical replay, security tests, and Playwright	Tests bind product behavior to internal contracts rather than provider SDK responses
Observability	Company-approved logs, metrics, and traces	EventPublisher uses provider-neutral trace, case, run, model, prompt, policy, and corpus identifiers

Portable core, company-specific first adapters

Provider agnosticism is a code-ownership rule, not a requirement to build several integrations. The first release implements only the MOHID/DeenTek-approved adapters. A provider SDK may appear only inside its adapter, and every adapter must pass a reusable contract suite. A second provider is added only when migration, resilience, cost, or a second client creates a real need.

18.3 Test-Driven Delivery Contract

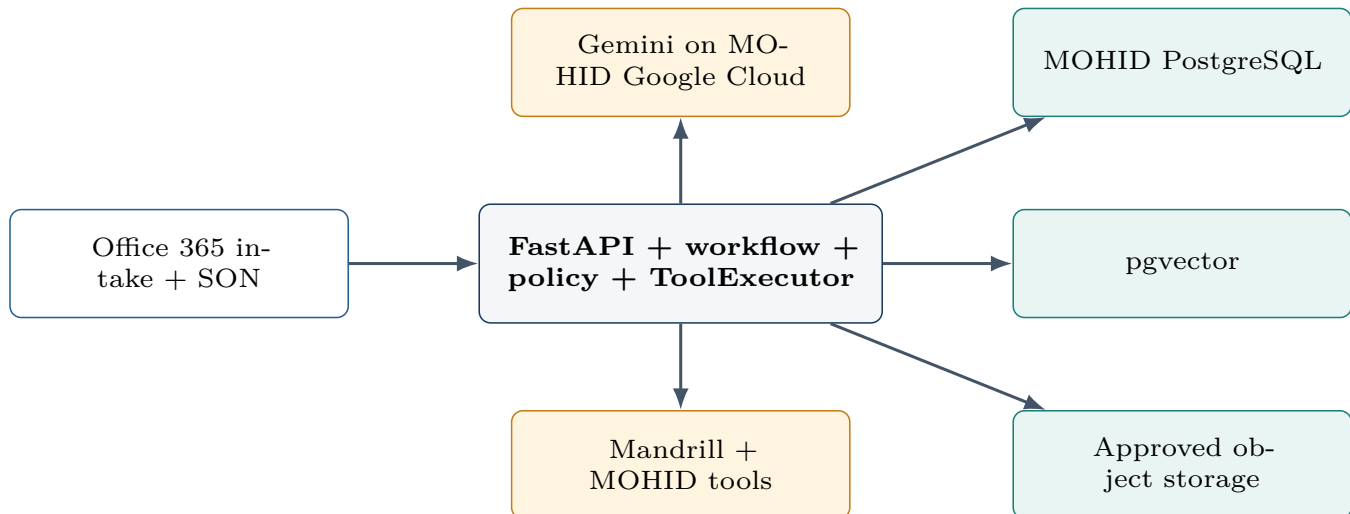
Every implementation increment follows the same loop:

1. write or select a failing acceptance, regression, or provider-contract test that expresses the required behavior;
2. implement the smallest domain behavior against local fakes;
3. run the same contract suite against the company-approved adapter in an authorized sandbox;
4. add integration tests across persistence, retrieval, workflow, and tool boundaries;
5. replay historical cases with all external actions disabled; and
6. promote only when the defined unit, contract, integration, security, evaluation, and end-to-end gates pass.

Test layer	Primary target	Release purpose
Unit	Domain invariants, normalization, policy, state transitions, chunking	Fast proof that core behavior is deterministic and provider-free
Provider contract	Model, embedding, vector, object, knowledge source/query, identity, inbound, outbound, and tool adapters	Proves fakes and company adapters obey the same typed semantics
Integration	PostgreSQL transactions, pgvector filters, workflow checkpoints, outbox, and receipts	Finds boundary failures and recovery defects
Regression/replay	Historical MOHID cases and known failure slices	Prevents previously observed wrong-source, incomplete, duplicate, and lost-state behavior
Security	Tenant isolation, webhook/auth validation, permissions, secrets, redaction, and injection cases	Blocks unsafe provider or data-boundary behavior

Test layer	Primary target	Release purpose
End-to-end	Office 365 test intake through SON review to Mandrill-approved test recipient	Demonstrates the actual company stack without enabling uncontrolled customer action

18.4 Component Topology



18.5 Required Provider Interfaces

Interface	Minimum responsibility
ModelProvider	Typed generation/analysis request; model/profile version; timeout; usage; redacted receipt; no tool authority
EmbeddingProvider	Batch embeddings; model/dimension/version metadata; deterministic error handling
VectorStore	Upsert/delete/search by tenant, corpus, document, and metadata; namespace isolation; health check
ObjectStore	Put/get versioned raw snapshots and artifacts; content hash; retention metadata
CaseRepository	Transactional cases, runs, decisions, actions, outcomes, and optimistic concurrency
KnowledgeSourceProvider	Discover approved sources; fetch source documents and metadata; list changes or synchronize from a checkpoint; return protocol-neutral acquisition results for the knowledge compiler
KnowledgeQueryProvider	Search approved knowledge; retrieve an exact document; return source metadata and version information; production implementation and live runtime use are deferred for the mTAP slice
InboundMessageProvider	Validate and normalize Office 365 or approved forwarding payloads; preserve message/thread identity; expose no provider-specific object to the core
OutboundMessageProvider	Send only authorized payloads through Mandrill; return stable request, delivery, failure, and reconciliation receipts
ToolExecutor	Validate an already authorized request envelope; dry-run where supported; execute, reconcile, and return a typed provider result and receipt; it does not decide support policy
EventPublisher	Publish audit/operational events after durable commit; support trace correlation
IdentityProvider	Authenticate human/service identity and return stable actor, tenant, roles, and claims

18.6 Environment Model

- **Local:** synthetic/redacted cases; fake providers that pass the shared contract suites; no network side effects.
- **Development:** MOHID-approved Google Cloud/database sandbox, test mailbox, Mandrill test recipients, and

manual plus automated integration tests.

- **Staging:** production-like company providers and policies; only approved test recipients; shadow or universal review.
- **Production:** MOHID-owned or authorized credentials, backups, monitoring, rollback, and approved automatic categories.

Configuration is environment-specific and versioned. Secrets are not committed. Development shortcuts cannot silently activate in staging or production.

18.7 Repository Shape

```
apps/
  api/                # FastAPI routes and composition
  son/                # company-approved SON web client
applications/
  mohid-support/      # versioned declarative behavior
    manifest.yaml
    knowledge.yaml
    taxonomy.yaml
    routing.yaml
    autonomy.yaml
    providers.yaml
    templates/
    evaluations/
packages/
  domain/             # enums, schemas, invariants
  workflow/           # explicit state machine
  knowledge/          # acquisition, canonicalization, chunking
  retrieval/          # query plans, vector adapter, evidence
  verification/       # claim, procedure, scope checks
  policy/             # versioned deterministic policy
  tools/              # tool contracts and adapters
  providers/          # GCP, Office 365, Mandrill, storage
  persistence/        # repositories and migrations
  evaluation/         # datasets, replay, metrics
infra/
  docker/
  environments/
docs/
  decisions/
  runbooks/
tests/
  acceptance/
  unit/
  contract/
  integration/
  replay/
  security/
  evaluation/
e2e/
```

19 API Surface

The endpoints in this section are MOHID runtime and SON application interfaces. They do not imply access to a HelpDocs provider API and do not constitute an MCP layer.

19.1 API Conventions

- Base path `/v1`; JSON request and response bodies.
- Authenticated tenant and actor context comes from trusted middleware, not client-supplied tenant fields.
- Mutations accept `Idempotency-Key` and consequential case changes require `expected_case_version`.
- Errors use typed codes, human-readable summaries, retryability, and trace ID.
- Cursor pagination is used for queues and timelines.

- API and domain schemas are versioned independently from provider payloads.

19.2 Required Endpoints

Method	Path	Purpose
POST	/v1/cases/intake	Create or idempotently acknowledge a manual/API case
GET	/v1/cases	List authorized queue cases with filters and cursor
GET	/v1/cases/{case_id}	Return case summary, active decisions, evidence, permissions, and version
GET	/v1/cases/{case_id}/timeline	Return ordered audit events
POST	/v1/cases/{case_id}/human-decisions	Approve, edit, reject, clarify, reroute, or escalate
POST	/v1/cases/{case_id}/replay	Create a non-dispatching Request Run against selected versions
POST	/v1/cases/{case_id}/reopen	Record reopen and create a new Request Run
POST	/v1/knowledge/syncs	Start controlled HelpDocs sync
GET	/v1/knowledge/syncs/{sync_id}	Return counts, errors, versions, and publish state
POST	/v1/evaluations/runs	Execute dataset replay with no external actions
GET	/v1/evaluations/runs/{run_id}	Return metrics, failures, and artifacts
GET	/v1/automation/controls	Read effective kill switches and category/tool enablement
PUT	/v1/automation/controls	Version and change authorized automation controls
POST	/v1/feedback	Attach validated feedback event

19.3 Manual Intake Example

```
POST /v1/cases/intake
Idempotency-Key: demo-ticket-0042

{
  "channel": "MANUAL",
  "external_thread_ref": "historical-0042",
  "requester_ref": "redacted-requester",
  "subject": "Kiosk ticket sales",
  "body_text": "Does the MOHID Kiosk support selling tickets?",
  "received_at": "2026-07-27T15:00:00-05:00"
}
```

19.4 Human Decision Example

```
POST /v1/cases/case_123/human-decisions

{
  "expected_case_version": 7,
  "action": "EDIT",
  "reason_code": "PROCEDURE_COMPLETENESS",
  "edited_response": "...",
  "route_target": null,
  "comment": "Added required prerequisite from cited HelpDocs."
}
```

The API returns the new case version and whether re-verification or another approval is required.

20 Engineering Epics and Build Order

20.1 Critical Path

Wave	Epics	Outcome
A	E01–E03	Repository, contracts, durable cases/audit, and versioned HelpDocs foundation
B	E04–E07	Evaluated retrieval, Evidence Packets, typed workflow, verification, and policy
C	E08–E10	Deterministic tools, SON, and accountable human decisions
D	E11–E12	Historical replay and live shadow evidence
E	E13–E14	Universal-review pilot followed by narrow automatic resolution
F	E15	Production hardening, runbooks, acceptance package, and handoff

E01 - Company-Aligned Repository, Test Harness, and Domain Contracts

Objective: Establish the approved repository, company environment targets, test harness, and stable V1 language before provider behavior is implemented.

Dependencies: Platform Living Reference; this paper; a closed V1 repository decision; access to the approved repository.

Outputs: Monorepo scaffold; domain package; enums and Pydantic/TypeScript schemas; semantic provider contracts and fakes, including the knowledge-source contract and deferred query seam; environment configuration model; CI; decision register; architecture record templates. No generic MCP interface or server is required.

Acceptance: Resolution, dispatch, routing, case/run, evidence, tool, human decision, event, and provider schemas validate; company integration targets are named; test and lint jobs run on a clean checkout before production adapter code is added.

Tests: Acceptance-test skeleton; schema round trips; invalid enum rejection; fake-provider contract suite; compatibility fixture; secret-scan; dependency and license checks.

Risks: Over-scaffolding before the vertical slice.

Definition of done: One command starts API/test services, contracts are versioned, and all later epics reference them rather than redefining fields.

E02 - PostgreSQL Case and Audit Model

Objective: Make Support Case, Request Run, state transitions, and audit events durable.

Dependencies: E01.

Outputs: Migrations; repositories; manual intake endpoint; optimistic concurrency; append-only audit API; transactional outbox skeleton.

Acceptance: Duplicate intake is idempotent; a case and run survive restart; concurrent human writes conflict safely; every transition creates an ordered event.

Tests: Migration up/down in disposable database; transaction rollback; idempotency; cross-tenant negative tests; event sequence and hash checks.

Risks: Treating event log as a substitute for queryable current state, or vice versa.

Definition of done: A manual case can be created, advanced through a no-action dry run, reconstructed from records, and queried by authorized tenant context.

E03 - HelpDocs Ingestion and Synchronization

Objective: Produce a versioned, structure-preserving MOHID corpus.

Dependencies: E01, E02, approved mTAP HTML pages or fixtures, and a corpus approver. Official HelpDocs API or export access is not required.

Outputs: KnowledgeSourceProvider; fake plus approved HTML/fixture adapter; discovery/fetcher; raw snapshots; canonical block parser; generated-Markdown scaffolder; block/chunk model; hashes; structural diff; approved corpus publication; sync report.

Acceptance: Source adapters pass one contract suite; five golden mTAP pages preserve headings, lists,

E03 - HelpDocs Ingestion and Synchronization (continued)

procedures, warnings, and URLs; Markdown and search projections derive from the same blocks; unchanged sync is idempotent; changed/removed/deprecated pages behave as specified; changed production content requires publication approval.

Tests: Source-provider contract; golden JSON/Markdown fixtures; source-fidelity checks; boilerplate removal; redirect/removal; partial-fetch failure; deterministic chunk IDs; structural-diff classification; corpus rollback.

Risks: Site HTML changes, screenshot-only steps, undocumented source ownership, or mistaking future API/MCP access for a replacement of the compiler.

Definition of done: A repeatable command builds a reviewable corpus candidate, records approval, publishes one immutable version, and explains every indexed chunk's origin.

E04 - Vector Database and Retrieval Evaluation

Objective: Retrieve relevant MOHID evidence reliably on real ticket language.

Dependencies: E03; initial labeled retrieval cases.

Outputs: EmbeddingProvider and VectorStore adapters; persistent index; query normalization; intent-aware queries; metadata filters; retrieval trace; section/procedure and justified context-expansion comparisons; benchmark report.

Acceptance: Relevant-source and active-context recall meet the provisional gate; the paired clean/noisy mTAP cases preserve expected evidence and disposition; retrieval never crosses corpus/tenant filters.

Tests: Adapter contract; upsert/delete; namespace isolation; clean/noisy pairs; query variants; provider timeout/quota; deterministic fixture evaluation.

Risks: Optimizing one score while losing procedure completeness; provider lock-in; tuning on the holdout set.

Definition of done: One versioned retrieval configuration is selected with measured evidence and known failure slices.

E05 - Evidence Packet Construction

Objective: Convert search candidates into defensible, per-intent evidence.

Dependencies: E04.

Outputs: Intent units; source spans; coverage; missing requirements; conflict records; packet persistence; packet API.

Acceptance: Multi-intent fixtures receive explicit coverage per intent; correct “helpful source” evidence cannot be ignored; insufficient or conflicted cases are represented without forced resolution.

Tests: Single/multi-intent; wrong-product near match; deprecated source; conflicting procedure; missing prerequisite; source locator round trip.

Risks: Packet becomes an unbounded context dump or hides disqualifying evidence.

Definition of done: A reviewer and downstream decision node can explain what is supported, missing, or conflicted without re-running retrieval.

E06 - Resolution, Dispatch, and Routing Workflow

Objective: Implement the explicit case state machine and three decision planes.

Dependencies: E02, E05.

Outputs: Workflow graph; checkpoint/recovery; typed decisions; route resolver; decision persistence; failure states.

Acceptance: Every fixture yields separate resolution, dispatch, and routing values; unsupported cases cannot resolve; replay does not execute tools; retry creates no duplicate decision side effects.

Tests: Allowed transition matrix; invalid combinations; clarification/reopen runs; crash/resume; routing without resolution mutation.

Risks: Encoding business policy in prompts or graph edges that cannot be versioned.

Definition of done: A case moves from intake to AWAITING_REVIEW or a denied dry-run outcome with a complete trace.

E07 - Verification and Policy Gate

Objective: Prevent unsupported, incomplete, conflicted, or unauthorized automatic action.

Dependencies: E05, E06.

Outputs: Claim extraction; claim-to-span mapping; procedure and terminology checks; deterministic policy evaluator; dispatch gate; kill switch.

Acceptance: Missing evidence, conflict, failed verifier, disabled category, or unavailable persistence prevents AUTOMATIC; policy decisions reproduce from recorded input/version.

Tests: Unsupported claim; missing step; wrong label; conflict; verifier outage; switch off/on; policy version rollback.

Risks: False assurance from a second model or overly permissive default.

Definition of done: No candidate reaches ToolExecutor without a persisted, reproducible dispatch decision and passing mandatory checks.

E08 - Deterministic Tool Layer

Objective: Execute only narrow, runtime-authorized, schema-valid actions.

Dependencies: E02, E07.

Outputs: ToolExecutor; tool registry; draft/send/route/escalate/ticket-prefill adapters; dry run; idempotency; receipts; reconciliation worker.

Acceptance: Unknown or failed receipt does not retry blindly; missing escalation destination cannot report success; duplicate request produces one external action.

Tests: Contract tests with fake providers; missing or mismatched authorization envelope; connector-auth-without-capability denial; timeout before/after provider acceptance; 429/5xx retry; invalid input; permission denial; reconciliation.

Risks: Provider semantics make “exactly once” impossible; use at-least-once processing with idempotent effects and reconciliation.

Definition of done: Staging can dry-run and execute approved test-recipient actions with complete receipts and no duplicates.

E09 - SON Case Queue and Case Detail

Objective: Make evidence, decisions, risk, ownership, and history usable by reviewers.

Dependencies: E02, E05–E07.

Outputs: SON web app in the DeenTek-approved frontend stack; queue filters; case header; request/evidence/verification/policy panels; timeline; role-aware technical drawer.

Acceptance: A reviewer can identify required action, inspect exact sources, understand blocked automation, and navigate without raw logs.

Tests: Component and end-to-end fixtures; empty/error/access-denied states; keyboard navigation; screen-reader labels; large timeline pagination.

Risks: Building analytics before core review or exposing sensitive provider payloads.

Definition of done: A non-developer can review a seeded case and correctly explain why it is in the queue.

E10 - Reviewer Actions and Human-Decision Logging

Objective: Make human review consequential, safe, and reconstructable.

Dependencies: E08, E09.

Outputs: Approve/edit/reject/clarify/reroute/escalate API and UI; diffs; reason codes; re-verification; concurrency handling.

Acceptance: Every action records actor, reason, case version, changes, and result; substantive edits cannot dispatch before verification; stale tabs cannot overwrite newer decisions.

Tests: Role matrix; optimistic conflict; edit/reverify; disabled action explanation; audit reconstruction.

Risks: Excessive forms increase handling time; reason taxonomy becomes punitive.

Definition of done: A complete reviewer session from open through approved test dispatch appears chronologically in the case record.

E11 - Historical Replay and Evaluation Harness

Objective: Make quality measurable before live action.

Dependencies: E03–E07; adjudicated cases.

Outputs: Ten-case bootstrap set; dataset schema; import/redaction; replay runner; expansion to the 50-case pre-pilot benchmark; metrics; severity; diff between versions; failure export.

Acceptance: Replays are side-effect free, version-pinned, reproducible, and report every required metric and failure slice; the 50-case benchmark is adjudicated before live shadow entry.

Tests: Dataset validation; deterministic mock-provider run; missing label; disputed case exclusion; regression detection.

Risks: Weak labels or tuning to a small benchmark.

Definition of done: A release candidate produces a signed-off evaluation report against development, regression, and holdout sets.

E12 - Shadow-Mode Deployment

Objective: Observe real or live-like requests without customer-facing action.

Dependencies: E01–E11; authenticated channel or approved import.

Outputs: Staging deployment; shadow intake; comparison workflow; monitoring; incident and rollback runbook.

Acceptance: All requests are persisted and processed; no external action is possible; actual human outcome can be compared; failures route to operations.

Tests: End-to-end shadow; channel replay; outage; kill switch; backup/restore; data-access review.

Risks: Treating shadow recommendations as ground truth or ingesting unapproved personal data.

Definition of done: A defined shadow sample is completed with mismatches adjudicated and no unapproved outbound action.

E13 - Universal-Review Pilot

Objective: Validate reviewer leverage and safe live dispatch with approval on every outbound action.

Dependencies: E12 exit; MOHID reviewer, routing, mailbox, and retention decisions.

Outputs: Production-like SON queues; approved channel adapter; reviewer training; support runbook; daily quality review.

Acceptance: Review and dispatch are reliable; reviewer usability is accepted; edit/override reasons are captured; stop/rollback procedures work.

Tests: Approved-recipient live test; failed dispatch; reviewer absence; queue aging; reopen/negative feedback; incident drill.

Risks: Review becomes a rubber stamp or adds more work than it removes.

Definition of done: Pilot report quantifies correctness, handling time, edit rate, routing, feedback, and readiness for any automatic category.

E14 - Limited Automatic-Resolution Pilot

Objective: Test one or two approved low-risk categories under strict policy and sampling.

Dependencies: E13 exit; approved thresholds, categories, sample rate, and owner.

Outputs: Category policy versions; automatic dispatch; post-resolution review; real-time stop metrics; expansion decision record.

Acceptance: All automatic cases pass gates, receipts reconcile, post-review sample meets thresholds, and no stop condition remains unresolved.

Tests: Boundary cases; category misclassification; policy disable; knowledge change; negative feedback; kill-switch drill.

Risks: Small samples hide rare severe failures; pressure to optimize automation rate.

Definition of done: Product owner and MOHID explicitly approve expansion, hold, narrow, or rollback based on evidence.

E15 - Production Hardening and Handoff

Objective: Make the accepted V1 operable, recoverable, and maintainable.

Dependencies: E12 minimum; E13/E14 as applicable.

Outputs: MOHID-approved server or Google Cloud deployment; backups; monitoring; alerts; SLOs; cost controls; dependency scan; operations/security runbooks; acceptance package.

Acceptance: Restore, rollback, kill-switch, provider outage, receipt reconciliation, and access review are demonstrated; owner and escalation paths are named.

Tests: Disaster-recovery drill; credential rotation; dependency failure; load and queue backpressure; audit export.

Risks: Infrastructure expands beyond validated scale or ownership remains ambiguous.

Definition of done: Named operators can deploy, monitor, pause, restore, investigate, and hand off V1 without relying on undocumented developer knowledge.

21 V1 Acceptance Criteria

V1 is accepted only when the following evidence exists.

21.1 Product and Workflow

- A support case travels end to end through intake, evidence, typed decisions, verification, policy, routing, review or automatic dispatch, outcome, feedback, and reopen.
- Resolution, dispatch, and routing remain independently queryable.
- SON supports every required reviewer action and exposes evidence before approval.
- One or more Request Runs can be reconstructed without overwriting earlier runs.

21.2 Knowledge and Evidence

- The published HelpDocs corpus is versioned, reproducible, and structure-preserving.
- Fake and approved HTML/fixture acquisition implementations pass the same KnowledgeSourceProvider contract, and provider transport details do not alter canonical document or evidence contracts.
- Every citation opens the correct source and locator.
- Changed, removed, deprecated, and failed-fetch behaviors pass.
- Multi-intent, insufficiency, and conflict fixtures produce correct Evidence Packets.

21.3 Governance and Safety

- Unsupported claims, policy uncertainty, unavailable verification, or failed persistence cannot auto-dispatch.
- Tenant/category/tool kill switches work and preserve review/shadow processing.
- Connector authentication alone cannot authorize a read or write; the runtime records the granted capability and policy decision before consequential use or execution.
- Tool actions are authorized, idempotent, receipt-validated, and recoverable.
- Human edits and overrides are attributable, version-checked, and re-verified when substantive.

21.4 Evaluation and Operations

- The adjudicated benchmark and holdout set are versioned.
- Offline and applicable live metrics satisfy approved gates.
- No unresolved Critical failure remains.
- Backup, restore, rollback, provider outage, and incident procedures are demonstrated.

- MOHID accepts the review/routing workflow and names operational owners.

21.5 Automatic Resolution

Automatic resolution is not required across all categories for V1 acceptance. If enabled, it must be limited to explicitly approved categories with category-specific evaluation, high post-dispatch sampling, stop conditions, and a recorded expansion/rollback decision.

22 Risks, Open Decisions, and Deferred Work

22.1 Risk Register

ID	Risk	Signal	Mitigation / owner
R-01	HelpDocs are incomplete for real cases	High escalation or missing-evidence rate	Knowledge-gap report; narrow categories; MOHID knowledge owner
R-02	Historical labels are weak or biased	Reviewer disagreement and unstable metrics	Dual adjudication for disputed/high-risk cases; dataset versioning
R-03	First-implementation scope expands	Epics start in parallel without vertical slice	Critical path; work-package exit gates; defer non-scope
R-04	Company provider access blocks integration	Missing Gemini/GCP, PostgreSQL, Graph, Mandrill, or hosting access	Contract-first local fakes; access checklist at E01; no alternate production stack
R-05	Review becomes rubber-stamping	Very fast approvals with later edits/reopens	Evidence-first UI; sampling; quality review; no reviewer score
R-06	Rare severe auto-resolution failure	Small benchmark and low incident count	Narrow categories; conservative gates; high sampling; kill switch
R-07	Office 365 or Mandrill integration is delayed	No Graph consent, mailbox ownership, sender approval, or Mandrill access	Keep provider-contract work moving locally; do not claim the integrated slice complete
R-08	Case data exceeds approved retention	Missing policy or unbounded raw payloads	Minimum data; redaction; retention decision before live use
R-09	Tool receipt is ambiguous	Timeouts after provider acceptance	UNKNOWN state; reconciliation; idempotency; no blind retry
R-10	Company environment differs from recorded assumptions	GCP project, server target, security standard, or provider path changes	Refresh stack checklist before E01; containerize; keep provider code inside adapters
R-11	Unknown HelpDocs API/export capability expands the first slice	Package A pauses for connector research or generic MCP work	Use approved HTML/fixtures behind KnowledgeSource-Provider; decide direct REST versus MCP only after the real capability and reuse value are known

22.2 Open Decisions by Deadline

ID	Decision	Needed by	Authority
O-01	Initial HelpDocs inventory, approval status, HTML/fixture acquisition method, and availability of an official API or export; direct REST versus MCP is decided only after the actual capability is known	E03 publish for inventory/acquisition; post-mTAP for optional connector choice	Akbar + MOHID knowledge owner
O-02	Historical ticket access, redaction, adjudicators, and benchmark labels	E11	Akbar + MOHID
O-03	MOHID-approved PostgreSQL/pgvector and object-storage targets, versions, regions, and owners	Before E02/E03 integration	Akbar + MOHID IT
O-04	Approved Gemini model/embedding endpoints, quotas, and benchmark candidates	Before E04 exit	Akbar + MOHID IT
O-05	Level 1, Level 2, specialist, manager queues and owners	Before E12	MOHID operations
O-06	Microsoft Graph mailbox/scopes/consent plus Mandrill sender, credentials, test recipients, and escalation target	Before live shadow/test send	MOHID IT/operations
O-07	Retention, redaction, access, and incident policy	Before live data	MOHID + Akbar
O-08	Approved evaluation thresholds and auto-eligible categories	Before E14	Product owner + MOHID
O-09	MOHID server or Google Cloud deployment target, Gemini/Vertex access, networking, secrets, cost, and operations owner	Before integrated environment	MOHID + Akbar
O-10	Acceptance authority and contract completion evidence	Before E13	Akbar + MOHID
O-11	Create a new V1 repository or use an isolated branch/new root in an approved existing repository; define what is ported from the legacy baseline	Before E01 begins	Akbar
O-12	Which ticket, form, escalation, or other external actions require explicit customer confirmation before submission	Before the affected ToolAction is enabled	MOHID operations + product owner

22.3 Deferred Backlog

Second-tenant administration, self-service workflow building, voice, kiosk integration, mobile, connector marketplace, generic MCP infrastructure, MCP hosting/discovery/administration, live HelpDocs querying, HelpDocs write-back, billing, broad analytics, arbitrary privileged actions, complex organization modeling, and commercial packaging remain outside V1. They require demonstrated MOHID need and a new roadmap decision.

23 Decision Register

ID	Decision	Status	Rationale
D-001	The earlier repository is limited to a technology baseline, fixture source, and regression reference	Accepted	Keep the first implementation focused on the current company-aligned solution
D-002	Support Case is the human-facing unit; Request Runs are versioned attempts	Accepted	Maintains one accountable lifecycle without overwriting history
D-003	Resolution, dispatch, and routing are separate contracts	Accepted	Locked Vision invariant

ID	Decision	Status	Rationale
D-004	Automatic resolution remains in V1 but follows replay, shadow, and universal review	Accepted	Tests the original product objective safely
D-005	PostgreSQL is the source of truth; vector search is a derived projection	Accepted	Durable relational cases, decisions, audit, and recovery
D-006	Structure-preserving canonical HelpDocs replace flattened text ingestion	Accepted	Required by documented regression cases and source traceability
D-007	Persistent vector storage begins with the vertical slice	Accepted	Avoids another disposable retrieval baseline
D-008	PostgreSQL plus pgvector is the proposed first VectorStore implementation	Provisional	Fits MOHID servers or Google Cloud while keeping the VectorStore contract portable
D-009	An explicit typed state machine is required; LangGraph is optional	Accepted	Workflow semantics and tests must remain independent of an orchestration library
D-010	Gemini through MOHID-owned Google Cloud is the first ModelProvider implementation	Accepted	Aligns the first integrated build with the company's approved environment
D-011	Office 365 inbound and Mandrill outbound are the first channel adapters	Accepted	Matches MOHID's stated mailbox and transactional-email stack
D-012	SON queue and case detail are required V1 surfaces	Accepted	Meaningful review and unified accountability require them
D-013	Policy begins as versioned typed code/configuration, not OPA	Accepted	Keeps first slice inspectable without weakening the gate
D-014	No external automatic action before Phase 4	Accepted	Progressive rollout and evidence requirement
D-015	Test-driven delivery and reusable provider-contract suites are mandatory	Accepted	Keeps the company-specific first stack replaceable without speculative duplicate integrations
D-016	Preserve the legacy repository as a historical baseline until O-11 closes	Accepted	Prevents accidental in-place drift and protects regression evidence
D-017	MOHID behavior is a declarative, versioned application package executed by the shared runtime	Accepted	Keeps tenant behavior configurable without permitting arbitrary graphs, executable package code, or policy-floor bypass
D-018	mTAP is the first bounded slice; KIOSK is the first reuse slice	Accepted	Fahad's logs provide a clean/noisy pair, expected evidence, related sources, provider faults, and correction failure for mTAP
D-019	Markdown is generated from canonical structured HelpDocs rather than serving as the sole source of truth	Accepted	Preserves block semantics and keeps human review and search projections consistent

ID	Decision	Status	Rationale
D-020	Development starts with ten redacted fixtures; the 50-case benchmark is required before live shadow mode	Accepted	Starts engineering from real evidence without treating full adjudication as a prerequisite to initial code
D-021	The immediate plan uses three acceptance-gated work packages rather than a fixed ten-day commitment	Accepted	Scope and timing depend on client access, adjudication, provider sandboxes, and corpus conditions
D-022	External APIs supply underlying capabilities; MCP is an optional adapter beneath semantic provider contracts, and generic MCP infrastructure is deferred	Accepted	Keeps source acquisition, live query, knowledge writing, and operational actions separately governed without expanding the mTAP proof

24 Immediate Vertical-Slice Plan

24.1 Plan Rule and End-to-End Goal

[V1 DESIGN] The first implementation is organized as three acceptance-gated work packages. They are a build order, not a dependable ten-working-day promise. Calendar estimates are set only after repository access, approved HelpDocs pages, provider sandboxes, and adjudication ownership are confirmed.

replayed or approved test intake → PostgreSQL case/run → versioned mTAP corpus → persistent retrieval → Evidence Packet → typed decisions → minimal SON case view → human decision audit → Mandrill test receipt

No customer-facing email and no automatic dispatch occur during these work packages. The technical proof is complete only when the mTAP slice crosses the full governed lifecycle. The value proof remains a comparison against the legacy baseline and human workflow on the same cases.

24.2 Acceptance-Gated Work Packages

Package	Work and output	Exit gate	Primary dependency
A - Baseline and knowledge compiler	Close O-11; scaffold semantic contracts, fakes, CI, and migrations; implement a fake and approved HTML/fixture KnowledgeSourceProvider; load ten redacted regression fixtures; curate roughly 8–15 related mTAP pages; create five golden parser fixtures; implement raw snapshots, canonical JSON, generated Markdown, hashes, structural diff, and one unpublished then approved corpus version.	Clean checkout runs; application package hydrates; source adapters pass one contract suite; cases/runs survive restart; unchanged sync is idempotent; golden pages preserve material structure; every chunk traces to a canonical block and URL; corpus publication has recorded approval.	Repository, approved mTAP pages or fixtures, knowledge owner, PostgreSQL target; no HelpDocs API prerequisite
B - Retrieval and governed decision proof	Benchmark section/procedure chunks and any justified parent-context or hybrid candidate; implement pgvector traces, intent units, Evidence Packets, case-aware correction runs, typed resolution/dispatch/routing, bounded drafting, verification, and policy-gated dry run.	Clean/noisy mTAP pair retains expected evidence and disposition; required related sources enter active evidence; correction feedback reuses the case; unsupported evidence cannot RESOLVE; every outcome is replayable with no side effect.	Expected sources/outcomes, adjudicator, Gemini/embedding sandbox

Package	Work and output	Exit gate	Primary dependency
C - SON and company-stack integration	Build minimal SON queue/detail and reviewer actions; persist actor, reason, diff, and timeline; connect approved Office 365 test intake and Mandrill test-recipient delivery; reconcile provider receipts and classify provider, channel, and answer-quality failures separately.	A reviewer can inspect and decide a seeded case; approved test input reaches a reconciled test receipt; quota and delivery failures fail safely; no path exists to an unapproved recipient or automatic dispatch. Access blockers remain named blockers, not a passed integration.	Graph consent/mailbox, Mandrill sender/test recipient, approved host, reviewer and route owner

24.3 First-Slice Acceptance

- A clean checkout can run tests and start API/SON.
- The MOHID application package validates, hydrates, and persists one immutable compatibility tuple per RequestRun.
- Fakes and each enabled company provider pass the same reusable contract suites.
- The fake and approved HTML/fixture source adapters pass the same KnowledgeSourceProvider suite; neither API nor MCP payloads appear in canonical document, Evidence Packet, decision, or case contracts.
- Ten fixtures enter idempotently and survive restart.
- Roughly 8–15 approved mTAP pages publish as one versioned corpus with exact URLs and structural approval; the final count follows the actual category boundary.
- Every indexed chunk traces to a heading path and content hash.
- Five golden parser pages preserve every material instruction, warning, order, link, and heading relation.
- The ten bootstrap cases produce stored retrieval traces and Evidence Packets.
- The clean and noisy mTAP setup cases retain the same expected intent, source, and disposition within approved variance.
- Required related mTAP evidence enters the Evidence Packet rather than remaining a “helpful sources” display.
- A correction such as “this is not the right answer” creates a case-aware retry, clarification, or escalation rather than an unrelated search.
- Model quota, retrieval quality, verification, and channel delivery failures are recorded as different failure classes.
- SON displays the request, evidence, typed decisions, and ordered audit timeline.
- A human decision records actor, reason, diff, and case version.
- Office 365 test intake and Mandrill-approved test sending either pass in the authorized sandbox or remain explicitly blocked by a named access prerequisite.
- The system has no path to an unapproved customer recipient or automatic dispatch.

24.4 Client Dependencies Beside the Plan

Dependency	Required confirmation or access	If missing
Knowledge	Approved mTAP page inventory or fixtures, HTML acquisition permission, source status, structural-diff approver, and confirmation of whether an official API/export exists	Package A can still prove the compiler with approved HTML/fixtures; it cannot publish a production-candidate corpus without source approval, and optional connector selection remains open
Case truth	Redacted full email bodies, expected source/evidence, expected disposition, and an adjudicator for disputed cases	Baseline remains provisional and value claims are blocked
Gemini/GCP	Project, endpoint, model and embedding access, quotas, regions, cost owner, and failure policy	Provider integration and performance claims remain blocked; contract fakes only
Data and hosting	Approved PostgreSQL/pgvector, object storage, server or GCP target, secrets, networking, backup, and operator	Integrated durability, security, and recovery gates remain blocked
Office 365	Shared mailbox owner, Graph consent/scopes or approved forwarding path, test messages, and thread identifiers	Package C uses replay only and cannot claim channel integration
Mandrill	Sender identity, credentials, approved test recipients, templates, domain/authentication status, and receipt semantics	Package C cannot pass outbound integration or deliverability gates
Operations	L1/L2/specialist routes, reviewer, retention policy, incident owner, and acceptance authority	Shadow/pilot entry and routing-value claims remain blocked

24.5 Work-Package Non-Goals

Do not build production-wide mailbox migration, unrestricted Mandrill sending, a complete policy engine, automatic dispatch, live HelpDocs querying, HelpDocs write-back, a production MCP server, generic MCP hosting/discovery/administration, a connector marketplace, broad analytics, voice, kiosk hardware integration, multi-tenant administration, Kubernetes, a second provider per boundary, or a polished management dashboard. No Package A, B, or C exit gate expands solely to support MCP.

24.6 Expansion Trigger

The first slice closes only after Packages A–C pass and a written failure review distinguishes technical correctness, provider reliability, and reviewer value. KIOSK ticket-sale and upgrade cases are then added as the first reuse slice without changing the canonical runtime lifecycle. Donation receipt follows for exact procedural completeness; program registration follows for clarification and escalation. The benchmark reaches 50 adjudicated cases before live shadow mode.

25 Closing Build Contract

MOHID V1 is not complete when a model returns a plausible reply. It is complete when the system can show what evidence was used, what resolution was proposed, whether execution was allowed, who owned the exception, what the human changed, what tool actually ran, how long each state lasted, and whether the outcome held.

Implementation anchor

Build the smallest end-to-end MOHID workflow on the company-approved stack, drive it through tests, keep provider dependencies behind owned contracts, and require evidence, policy, review, replay, and measured results before automation expands.

One case. One governed lifecycle. One defensible outcome.

A GitHub Issue Template

Title: [E##] <specific deliverable>

Objective

- What product or engineering outcome does this issue create?

Dependencies

- Prior issue, decision, credential, dataset, or environment.

Contract changes

- Schema/API/event/policy changes, or "none".

Implementation

- Concrete files, migrations, screens, adapters, or jobs.

Acceptance criteria

- Observable pass/fail statements.

Tests

- Unit, integration, contract, security, evaluation, or end-to-end.

Risks and failure behavior

- How the feature fails closed and how it is recovered.

Evidence

- Logs, screenshots, metric report, migration output, or replay artifact.

Definition of done

- Code reviewed, tests pass, docs/decision records updated, artifact linked.

B Initial Reason-Code Vocabulary

Code	Meaning
EVIDENCE_SUFFICIENT	Approved evidence covers the required intent or action
EVIDENCE_INSUFFICIENT	Required evidence is missing
EVIDENCE_CONFLICT	Approved sources materially conflict
ACTIVE_EVIDENCE_MISSING	A required retrieved source did not enter the Evidence Packet
PROCEDURE_INCOMPLETE	Prerequisite, ordered step, warning, or completion condition is missing
UNSUPPORTED_CLAIM	Proposed substantive claim lacks approved support
CORRECTION_RECEIVED	Requester feedback requires a case-aware retry, clarification, reopening, or escalation
MISSING_REQUEST_FIELD	Clarification is required for an explicit field
ACCOUNT_ACCESS_REQUIRED	Resolution needs account or internal-system access
SPECIALIST_JUDGMENT	Higher-authority domain judgment is required
CATEGORY_REVIEW_ONLY	Current rollout policy requires human approval
CATEGORY_AUTO_ENABLED	Category is enabled for automatic dispatch subject to all other gates
TOOL_NOT_ALLOWED	Requested action is outside the tool allowlist or actor permission
TOOL_INPUT_INVALID	Required typed input is missing or invalid
TOOL_RECEIPT_UNKNOWN	Provider outcome is indeterminate and requires reconciliation
MODEL_PROVIDER_UNAVAILABLE	Model quota, timeout, or outage prevented the configured model step
CHANNEL_DELIVERY_FAILED	Authorized response could not be confirmed as delivered by the outbound provider
CORPUS_APPROVAL_REQUIRED	A validated structural change is awaiting knowledge-owner publication approval
POLICY_DENIED	Deterministic policy prohibits execution
KILL_SWITCH_ACTIVE	Automatic dispatch is paused
ROUTE_LEVEL_1	Assign to central or Level 1 queue
ROUTE_LEVEL_2	Assign to Level 2 queue
ROUTE_SPECIALIST	Assign to configured specialist
ROUTE_MANAGER	Assign to manager or policy-exception owner
HUMAN_CORRECTION	Reviewer corrected a substantive proposal
KNOWLEDGE_GAP	Correct resolution requires missing or outdated HelpDocs